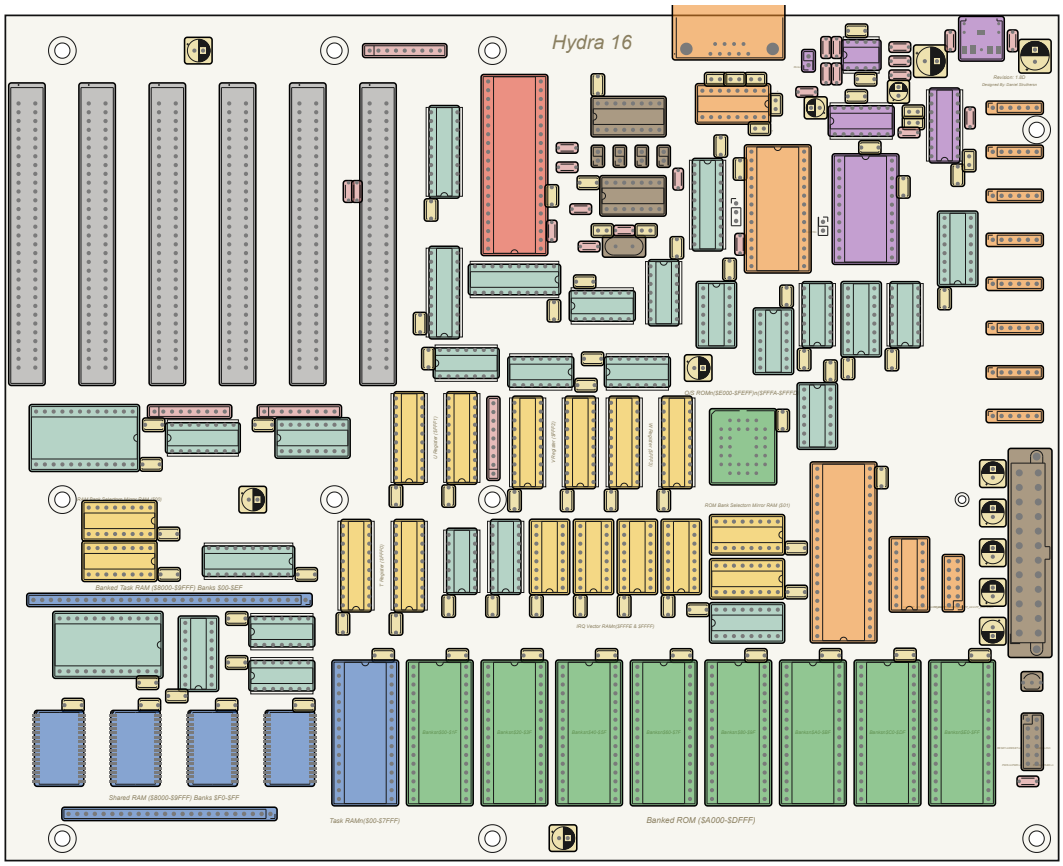
THE HYDRA PROJECT

HYDRA-16

Technical Reference



Every keyword, word, form, command and system call:
its syntax, its exact behaviour, its limits and its errors

HydraOS 1.0 · preview

About This Reference

This is the *Hydra-16 Technical Reference*, **in progress**. Its first chapter, HyBASIC's keywords, is being written letter by letter; the other chapters have one or two entries each so far, to show what they will be.

The *User's and Builder's Manual* teaches the Hydra, and its appendices list what each language has in a line apiece. This book is where to look when a line isn't enough: when the question is what a keyword does with an empty string, which errors a call can return, or what is left in a variable after a loop ends. It doesn't teach. It assumes the manual has been read, and it says the same thing about every entry, in the same order.

What it will hold

CHAPTER	ENTRIES	WRITTEN SO FAR
1. HyBASIC keywords	Every statement, function and operator	`ABS` to `PUT` (all of A to P, but the graphics keywords, which will be written together with `SCREEN`)
2. Commands	The shell's own commands, and the programs in `/bin`	`cp`
3. System calls	Every call a program can make	`OPEN`, `READ`
4. HyForth words	The core and every library	`>number`
5. hylang forms	The special forms and the functions	`let`

The other reference appendices of the manual (the 65C02's instructions, the character sets, the memory map, the errors) will follow in the same way.

How to read an entry

Every entry has the same parts, in this order. A part with nothing to say is left out.

`Syntax`
: Each form the thing can take. Words in `CAPITALS` (or, for a command or a HyForth word, as printed) are typed as they are; *italic* names stand for what you supply; `[ ]` encloses what can be left out; `{ | }` is a choice of one; `...` means the last item can repeat.

`Arguments`
: Each one: its type, its range, and what it means. For a system call, the registers in.

`Result`
: What a function returns, a word leaves on the stack, or a call leaves in the registers.

Description
: What happens, in the order it happens: what is evaluated and when, what is changed, where it stops.

Errors
: Each error it can raise, with its number and its message as the Hydra prints it, and what causes it.

Notes
: Limits, edge cases, and behaviour that differs from what another system's user would expect.

See also
: Related entries.

Example
: A transcript, taken from the Hydra. Every one in this book is replayed in the emulator when the book is built, so what is printed here is what the ROM does.

How it was written

Each entry was written from the interpreter's, the program's or the kernel's source, and each statement of behaviour in it was then tried on the system. Where the source and the trial disagreed with what one would expect of the language elsewhere (QuickBASIC, the Forth standard), the entry says what the Hydra does.

Section and chapter numbers such as "section 13.2" and "Appendix G" are the *User's and Builder's Manual*'s.

Contents

About This Reference 2

1 HyBASIC Keywords 7

ABS 7
ABSOLUTE 8
AND 8
APPEND 9
ARG$ 9
AS 10
ASC 10
ASM ... END ASM 11
ATN 13
BANK 14
BASE 15
BASE$ 16
BEEP 16
BINARY 17
BIT 17
BYE 17
CALL 18
CASE 19
CDBL, CSNG 19
CHDIR 20
CHR$ 20
CINT, CLNG 21
CIRCLE 22
CLEAR 23
CLOSE 24
CLS 24
COLOR 25
COMMAND$ 25
COMPLEX 26
CONST 27
CONT 28
COS 29
CSRLIN 29
CVN 30
DATA 31
DATE$ 32
DECLARE 32
DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR 33
DELETE 34
DENOMINATOR 34
DIGITS 35
DIM 36

DIR$ 38
DO ... LOOP 38
EDIT 40
ELSE, ELSEIF 40
END 40
ENV$, ENVIRON$ 41
ENVIRON 42
EOF 42
EQV 43
ERASE 44
ERL, ERR, ERR$ 44
ERROR 45
EXIT 46
EXP 47
FIB 47
FILES 48
FIX 49
FIXED 50
FOR ... NEXT 51
FRE 52
FREEFILE 53
FUNCTION ... END FUNCTION 54
GCD 55
GET 56
GOSUB 57
GOTO 58
HEX$ 59
IF 60
IMAG 61
IMP 62
INCLUDE 62
INKEY$ 64
INPUT 64
INPUT$ 66
INSTR 67
INT 68
IS 69
KILL 69
LBOUND, UBOUND 70
LCASE$, UCASE$ 71
LEFT$, RIGHT$ 71
LEN 72
LET 73
LINE INPUT 74
LIST 75
LOAD 76
LOC, LOF 77
LOCATE 78
LOG 78

LOOP 79
LTRIM$, RTRIM$ 79
MID$ 80
MKDIR, RMDIR 81
MKN$ 82
MOD 83
NAME 84
NEW 85
NEXT 85
NOT 85
NUMERATOR 86
OCT$ 86
ON 86
OPEN 88
OPTION 90
OR 91
OUTPUT 91
PEEK, POKE 91
PI 92
PLAY 93
POS 94
PRESERVE 95
PRINT 95
PUT 97

2 Commands 99
cp 99

3 System Calls 102
OPEN 102
READ 104

4 HyForth Words 107
>number 107

5 hylang Forms 109
let 109

CHAPTER 1

HyBASIC Keywords

HyBASIC compiles a program whole before it runs it: every line is read, every name resolved and every block matched, and only then does the first statement execute. An error the compiler finds (a `FOR` with no `NEXT`, a label that isn't there) is therefore reported before anything has run, with the line it was found on. A line typed at the prompt without a number is compiled and run the same way, as a program of one line.

An error is reported as its message, in small letters. `ERR` then holds its number, `ERR$` the message and `ERL` the line; the numbers given in each entry are `ERR`'s. A failure reported by the system (a disk that is full, a name that isn't there) has HyBASIC's own number where there is one that fits (53, file not found), and otherwise 256 plus the system's code (Appendix G).

A number is an integer of any length, a rational, a decimal or a complex number, and arithmetic on them is exact (Chapter 18). A string is a sequence of bytes, each 0 to 255, up to 64K of them. A string written in a program is enclosed in double quotes and can hold any character but the double quote itself: there is no doubled form for one inside a string. `CHR$(34)` is the way to make one.

ABS

A numeric function: the absolute value of a real number.

SYNTAX

```
ABS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an integer, a rational or a decimal.

RESULT

x if *x* is zero or more, and *–x* if it is less. The result is the same kind of number as *x*, and exact: an integer gives an integer, of whatever length; a rational gives a rational in lowest terms; a decimal gives a decimal with the same digits. There is no largest value and no overflow.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex. A complex number has no sign, and `ABS` does not give its modulus
13	`type mismatch`	*x* is a string
2	`syntax error`	No argument, or more than one

NOTES

For the modulus of a complex number *z*, use `SQR(REAL(z) ^ 2 + IMAG(z) ^ 2)`.

SEE ALSO

`SGN`, `INT`, `FIX`, `REAL`, `IMAG`.

EXAMPLE

```
> PRINT ABS(-7); ABS(7); ABS(-7/2); ABS(-2.5)
 7  7  7/2  2.5
> PRINT ABS(-2 ^ 100)
 1267650600228229401496703205376
> PRINT ABS(3+4i)
illegal function call
> PRINT ERR; SQR(REAL(3+4i) ^ 2 + IMAG(3+4i) ^ 2)
 5  5
```

ABSOLUTE

Part of `CALL ABSOLUTE`, which calls machine code at an address. See `CALL`.

AND

An operator: the bitwise AND of two integers, which is also the logical AND of two truth values.

SYNTAX

```
a AND b
```

ARGUMENTS

a, b

Numeric expressions whose values are real.

RESULT

An integer. Each bit of it is 1 where the same bit is 1 in both *a* and *b*, and 0 elsewhere.

DESCRIPTION

Both operands are evaluated, always: there is no short cut when the first is 0. Each is then rounded to the nearest integer, a half going to the even one (1.5 and 2.5 both become 2), and the two integers are combined bit by bit.

The integers can be of any length. A negative one is taken in two's complement with as many sign bits as are needed, so –1 is all ones, and `-1 AND x` is *x* for every integer *x*.

A comparison gives –1 for true and 0 for false. Since –1 is all ones and 0 is none, `AND` of two comparisons is their logical AND, and `IF a > 0 AND b > 0 THEN` means what it says. `AND` of a truth value and some other number is a bit operation still: `(1 = 1) AND 2` is 2, which counts as true, but `1 AND 2` is 0, which is false, though both of its operands count as true.

`AND` binds more tighter than `OR`, `XOR`, `EQV` and `IMP`, and less tightly than `NOT` and the comparisons: `NOT a AND b = c OR d` is `((NOT a) AND (b = c)) OR d`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An operand is a string
5	`illegal function call`	An operand is complex

SEE ALSO

`OR`, `XOR`, `NOT`, `EQV`, `IMP`, `BIT`, `SHL`.

EXAMPLE

```
> PRINT 12 AND 10; -1 AND 255; (1 < 2) AND (3 < 2); 1 AND 2
 8  255  0  0
> PRINT 2.5 AND 3; 1.5 AND 3; 2 ^ 40 + 5 AND 2 ^ 40 + 3
 2  2  1099511627777
> PRINT 0 AND (1 / 0)
division by zero
```

APPEND

A mode of `OPEN`: `OPEN path$ FOR APPEND AS #f` opens a file for writing at its end. See `OPEN`.

ARG$

A string function: one of the arguments the program was started with.

SYNTAX

```
ARG$(n)
```

ARGUMENTS

n

A numeric expression: which argument. 0 is the program's own name, 1 the first argument after it.

RESULT

The argument, as a string. The arguments are those of the command that started HyBASIC with a program to run: after `basic prog.bas one two`, `ARG$(0)` is `prog.bas`, `ARG$(1)` is `one` and `ARG$(2)` is `two`. An argument is passed whole, as the shell made it: one that was quoted there can hold spaces.

The empty string if there is no argument *n*: past the last one, for a negative *n*, and for every *n* when HyBASIC was started without a program (at its prompt).

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*n* is a string

NOTES

There is no count of the arguments. Read them until one is empty, or split `COMMAND$`.

SEE ALSO

`COMMAND$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; ARG$(0); "]["; ARG$(1); "]["; ARG$(2); "]["; ARG$(3); "]"' >args.bas
% basic args.bas one 'two words'
[args.bas][one][two words][]
```

AS

Part of four statements, where it introduces a name, a number or a type: `DIM v AS type`, `OPEN path$ FOR mode AS #f`, `NAME old$ AS new$`, and a field's line in `TYPE`. See those.

ASC

A numeric function: the code of a string's first byte.

SYNTAX

```
ASC(s$)
```

ARGUMENTS

s$

A string expression of one byte or more.

RESULT

An integer from 0 to 255: the value of the first byte of *s$*. The rest of the string is not looked at. A string holds bytes, not characters of any one set, so the result is the byte itself; which character the console draws for it depends on the character set in use (Appendix E). The codes 32 to 126 are ASCII's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is the empty string
13	`type mismatch`	The argument is a number

NOTES

`ASC` is the inverse of `CHR$` for one byte: `ASC(CHR$(n))` is *n* for every *n* from 0 to 255, including 0 (a zero byte doesn't end a string). To test a string that may be empty, test `LEN(s$)` first.

SEE ALSO

`CHR$`, `LEN`, `MID$`, `VAL`.

EXAMPLE

```
> PRINT ASC("A"); ASC("Abc"); ASC(" "); ASC(CHR$(0) + "x"); ASC(CHR$(255))
 65  65  32  0  255
> PRINT ASC("")
illegal function call
```

ASM ... END ASM

A block: 65C02 assembly language inside a program, assembled when the program is compiled.

SYNTAX

```
ASM
    assembly language lines
END ASM

CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

label

A label defined in one of the program's `ASM` blocks, spelled as it is there. The assembler's names keep their case: `Add1` and `add1` are two labels.

a, x, y

Numeric expressions: the values the A, X and Y registers hold as the routine is entered, a byte each. Each is 0 if it is left out.

DESCRIPTION

`ASM` alone on a line starts a block and `END ASM` alone on a line ends it. The lines between are not HyBASIC: they are in the language of the `as` assembler (Chapter 27), with its labels, directives and expressions. A program can have several blocks; they are assembled together, as one source, in the order they appear, so a label in one can be used in another.

The blocks are assembled when the program is compiled, before it runs. Their code goes into a RAM bank of its own, 8K long, which is at `$8000` to `$9FFF` while the code runs. Statements don't run "through" a block: when control reaches `ASM`, it goes on after `END ASM`.

`CALL ASM label` calls the code at *label* as a subroutine (`jsr`), which returns with `rts`. `RREG` then gives the registers it returned with.

The assembler is given some of the program's names as symbols. A global numeric variable's name is the address of its value: five bytes, the first its kind (0 for an integer) and the next four an integer's value, low byte first. An integer `CONST`'s name is its value. Each is defined in capitals and in small letters. The names `A`, `X` and `Y` are left out, since they are the registers'.

ERRORS

ERR	MESSAGE	CAUSE
90	`ASM:` and the assembler's message	An error in a block, reported at its line when the program is compiled
90	`ASM:` *label*`: no such label in an ASM block`	`CALL ASM` names a label that no block defines

NOTES

The code's bank is at `$8000` only while the code runs, so the code can't use `BANK`'s bank at the same time, and a `PEEK` of `$8000` from HyBASIC doesn't see the code.

A program can name at most 64 different labels in its `CALL ASM` statements.

The code runs as part of HyBASIC's task, with the whole machine open to it. A routine that doesn't return, or that writes where it shouldn't, takes HyBASIC down with it.

SEE ALSO

`CALL`, `SYS`, `RREG`, `PEEK`, `POKE`, `BANK`.

EXAMPLE

```
> 10 CALL ASM seven : RREG A : PRINT A
> 20 CALL ASM add1, 41 : RREG A : PRINT A
> 30 END
> 40 ASM
> 50 seven: lda #7
> 60        rts
> 70 add1:  inc a
> 80        rts
> 90 END ASM
> RUN
 7
 42
```

ATN

A numeric function: the arctangent of a real number.

SYNTAX

```
ATN(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The angle, in radians, whose tangent is *x*: more than $-\pi/2$ and less than $\pi/2$. `ATN(0)` is the integer 0. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed), whatever kind of number *x* is.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

`4 * ATN(1)` is the usual way to get π in a BASIC, but here it multiplies the rounding of `ATN(1)` by four. `PI` gives π to the digits set, directly.

SEE ALSO

`TAN`, `SIN`, `COS`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT ATN(1); ATN(0); ATN(-1); ATN(1/2)
 0.785398163397  0 -0.785398163397  0.463647609001
> DIGITS 30 : PRINT ATN(1) : DIGITS 12
 0.785398163397448309615660845820
```

BANK

A statement and a function: chooses, or reports, the RAM bank that `PEEK` and `POKE` see at `$8000`.

SYNTAX

```
BANK n
BANK()
```

ARGUMENTS

n

A numeric expression from 0 to 255: a RAM bank's number.

RESULT

`BANK()` is the number last set by `BANK`, or 0 if none has been.

DESCRIPTION

The addresses `$8000` to `$9FFF` are a window, 8K wide, onto one RAM bank at a time (Appendix F). `BANK n` sets the bank that a `PEEK` or `POKE` of an address in that range will use. It changes nothing else: the bank is put in the window for the moment of each `PEEK` and `POKE` and then taken out again, because HyBASIC keeps its own data in banks and uses the window itself.

HyBASIC takes RAM banks for its own use as it needs them, for the program's text, its strings and its arrays, and `BANK` doesn't check for them: a `POKE` into a bank HyBASIC is using damages it. A program that wants a bank to itself should take one high in the task's range, which HyBASIC reaches last if at all.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

n is not checked against the banks the task has. A task with three RAM modules has banks 0 to 47.

`BANK` has no effect on the bank an `ASM` block's code runs in.

SEE ALSO

`PEEK`, `POKE`, `SYS`, `ASM`.

EXAMPLE

```
> PRINT BANK() : BANK 40 : POKE &H8000, 65 : PRINT PEEK(&H8000); BANK()
 0
 65  40
> BANK 41 : POKE &H8000, 66 : BANK 40 : PRINT PEEK(&H8000)
 65
```

BASE

A statement: sets the base that numbers are written and read in.

SYNTAX

```
BASE b$
```

ARGUMENTS

b$

A string expression: a base string (sections 18.4 to 18.6). `"d"` is decimal, `"x"` hexadecimal, `"b"` binary, `"o"` octal; `"7r"` is base 7.

DESCRIPTION

From the next statement on, every number that HyBASIC converts to text is written in the base, and every number it converts from text is read in it. That covers `PRINT`, `WRITE`, `STR$` and `PRINT #`; `INPUT`, `READ` and `VAL`; and the numbers written in the program's own lines that are compiled after the statement has run, which at the prompt means the lines typed after it.

A number with a prefix says its own base whatever the setting: `&HFF` and `&B1010` are 255 and 10 in every base.

The setting stays until another `BASE`, and is `"d"` when HyBASIC starts.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*b$* isn't a base string
13	`type mismatch`	*b$* is a number

NOTES

`OPTION BASE`, which sets the lowest index of arrays, is a different statement: see `OPTION`.

To write one number in another base without changing the setting, use `HEX$`, `OCT$`, or `STR$` with a base string.

SEE ALSO

`BASE$`, `DIGITS`, `HEX$`, `OCT$`, `STR$`, `VAL`.

EXAMPLE

On the second line the 10 and the 6 are read in hex, as sixteen and six, and their sum, twenty-two, is written in hex:

```
> X = 255 : BASE "x" : PRINT X
 FF
> PRINT 10 + 6; BASE$
 16 x
> BASE "d" : PRINT X
 255
```

BASE$

A string function: the base string that `BASE` last set.

SYNTAX

```
BASE$
```

RESULT

The string given to the last `BASE` statement, as it was given; `d` if there has been none.

SEE ALSO

`BASE`.

EXAMPLE

```
> PRINT BASE$ : BASE "16r" : PRINT BASE$ : BASE "d"
d
16r
```

BEEP

A statement: sounds the terminal's bell.

SYNTAX

```
BEEP
```

DESCRIPTION

Writes the byte 7 (BEL) to standard output. A terminal sounds its bell, or flashes its window, as it is set to. If output has been sent to a file, the byte goes into the file.

SEE ALSO

`SOUND`, `PLAY`.

BINARY

A mode of `OPEN`: `OPEN path$ FOR BINARY AS #f` opens a file for reading and writing bytes at any position. See `OPEN`, `GET` and `PUT`.

BIT

A numeric function: one bit of an integer.

SYNTAX

```
BIT(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real. It is rounded to the nearest integer, a half going to the even one.

n

A numeric expression: which bit. Bit 0 is the lowest, worth 1; bit *n* is worth 2 to the power *n*.

RESULT

1 if bit *n* of *x* is set, and 0 if it is clear.

A negative *x* is taken in two's complement with endless sign bits, so every bit of −1 is 1, however high. 0 if *n* is negative.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

SEE ALSO

`AND`, `OR`, `XOR`, `SHL`, `SHR`.

EXAMPLE

```
> PRINT BIT(5, 0); BIT(5, 1); BIT(5, 2); BIT(-1, 100); BIT(2 ^ 70, 70)
 1  0  1  1  1
```

BYE

A statement: leaves HyBASIC. `BYE [n]` is another spelling of `SYSTEM [n]`. See `SYSTEM`.

CALL

A statement: calls a `SUB`, machine code at an address, or a label in an `ASM` block.

SYNTAX

```
[CALL] name [(] [argument [, argument] ...] [)]
CALL ABSOLUTE (address)
CALL ASM label [, a [, x [, y]]]
```

ARGUMENTS

name

The name of a `SUB` defined anywhere in the program.

argument

An expression, one for each of the `SUB`'s parameters, of the parameter's type.

address

A numeric expression, 0 to 65,535: where the machine code is.

DESCRIPTION

A `SUB`. The word `CALL` is optional, and so are the parentheses round the arguments: `Show "x", 2`, `CALL Show("x", 2)` and `CALL Show "x", 2` are the same call. The `SUB` runs, and the program goes on with the statement after the call.

An argument that is a variable, or an array's element, is passed **by reference**: the parameter is another name for it, and an assignment to the parameter changes it. Any other expression is passed by value. To pass a variable by value, put it in parentheses of its own, which makes it an expression: `Inc (X)`.

`CALL ABSOLUTE` calls the machine code at *address* as a subroutine, with the A, X and Y registers 0. It is `SYS address` by QuickBASIC's name. `RREG` gives the registers afterwards. See `SYS`.

`CALL ASM` calls a label of an `ASM` block. See `ASM`.

ERRORS

ERR	MESSAGE	CAUSE
35	`subprogram not defined`	No `SUB` has that name. Found when the program is compiled
37	`argument-count mismatch`	More arguments, or fewer, than the `SUB` has parameters
13	`type mismatch`	A string where the parameter is a number, or a number where it is a string

NOTES

A `SUB` can be called before the line that defines it: the whole program is compiled before any of it runs.

A `FUNCTION` is not called with `CALL`. It is used in an expression.

SEE ALSO

`SUB`, `FUNCTION`, `DECLARE`, `SYS`, `ASM`, `RREG`.

EXAMPLE

```
> 10 SUB Inc (N) : N = N + 1 : END SUB
> 20 X = 1 : Inc X : PRINT X
> 30 CALL Inc((X)) : PRINT X
> 40 DIM A(3) : A(1) = 5 : CALL Inc(A(1)) : PRINT A(1)
> RUN
 2
 2
 6
```

CASE

Part of `SELECT CASE`: `CASE` starts each of its choices. See `SELECT`.

CDBL, CSNG

Numeric functions: their argument, unchanged.

SYNTAX

```
CDBL(x)
CSNG(x)
```

ARGUMENTS

x

A numeric expression.

RESULT

x. In QuickBASIC these convert to double and to single precision. HyBASIC has one kind of numeric variable, which holds any number exactly, so there is nothing to convert: a rational stays a rational, and no digits are lost or gained. They are here so that programs written for QuickBASIC run.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

`CINT`, `FIXED`, `RATIONAL`, `DIGITS`.

EXAMPLE

```
> PRINT CDBL(1/3); CSNG(2.5); CDBL(7)
 1/3  2.5  7
```

CHDIR

A statement: changes the current directory.

SYNTAX

```
CHDIR path$
```

ARGUMENTS

path$

A string expression: the directory's name. One that doesn't start with `/` is taken from the current directory.

DESCRIPTION

Sets the current directory of HyBASIC's task, which is where a file name that doesn't start with `/` is looked for by `OPEN`, `LOAD`, `SAVE`, `FILES`, `KILL` and the rest. The change lasts until the next `CHDIR` or until HyBASIC ends. It doesn't change the directory of the shell that started HyBASIC.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name
76	`path not found`	The name is a file, not a directory
13	`type mismatch`	*path$* is a number

SEE ALSO

`DIR$`, `FILES`, `MKDIR`, `RMDIR`, `SHELL`.

EXAMPLE

```
> CHDIR "/ram" : OPEN "note.txt" FOR OUTPUT AS #1 : CLOSE : FILES
bin/            lib/            note.txt
> CHDIR "/nothere"
file not found
> CHDIR "/rom/bin/ls"
path not found
```

CHR$

A string function: a string of one byte.

SYNTAX

```
CHR$(n)
```

ARGUMENTS

n

A numeric expression from 0 to 255. One that isn't an integer is rounded to the nearest, a half going to the even one.

RESULT

A string one byte long, whose byte is *n*. Any value can be made, 0 included, and a string can hold any of them anywhere: `LEN(CHR$(0))` is 1.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 0 or more than 255
13	`type mismatch`	*n* is a string

NOTES

What the console shows for a byte depends on the byte and on the character set (Appendix E). The codes below 32 are controls to a terminal: `CHR$(7)` rings its bell, `CHR$(10)` starts a new line, and `CHR$(27)` begins an escape sequence (section 13.7).

SEE ALSO

`ASC`, `STRING$`, `SPACE$`.

EXAMPLE

```
> PRINT CHR$(72) + CHR$(105); LEN(CHR$(0)); ASC(CHR$(65.5)); ASC(CHR$(66.5))
Hi 1  66  66
> PRINT CHR$(256)
illegal function call
```

CINT, CLNG

Numeric functions: a real number rounded to the nearest integer.

SYNTAX

```
CINT(x)
CLNG(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The integer nearest to *x*. A value exactly half way between two integers goes to the even one: 2.5 gives 2, 3.5 gives 4, and −2.5 gives −2. This is the rounding QuickBASIC's `CINT` does.

The two functions are the same. An integer here can be of any length, so there is no range to pass, where QuickBASIC's `CINT` overflows beyond 32,767.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`INT` (rounds down), `FIX` (rounds toward zero), `FIXED`.

EXAMPLE

```
> PRINT CINT(2.5); CINT(3.5); CINT(-2.5); CINT(7/2); CINT(2.4); CINT(-2.6)
 2  4 -2  4  2 -3
> PRINT CLNG(2 ^ 40 + 1/2)
 1099511627776
```

CIRCLE

A statement: draws a circle, or a filled disc, on the graphics screen.

SYNTAX

```
CIRCLE [STEP] (x, y), r [, c [, start [, end [, aspect [, F]]]]]
```

ARGUMENTS

x, y
: Numeric expressions: the centre. With `STEP`, they are an offset from the last point referenced.

r
: A numeric expression, 0 or more: the radius.

c
: A numeric expression, 0 to 255: the colour. A mode with fewer colours uses the number's low bits: `SCREEN 1` has four colours, so 5 draws in colour 1. If it is left out, the pen's colour, which `COLOR` sets.

start, end, aspect
: Accepted, evaluated and ignored. QuickBASIC draws arcs and ellipses with them; HyBASIC always draws a whole circle.

F
: The letter F: fill the circle, making a disc.

DESCRIPTION

Needs a graphics mode (`SCREEN` 1 or more). The coordinates and the radius are in pixels, each rounded to the nearest, unless `WINDOW` has set coordinates of the program's own; then the radius is in the window's x units. The centre becomes the last point referenced, where a following `STEP` or `LINE -(x, y)` starts

from.

A radius of 0 draws the centre's one dot.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No graphics mode is set; or *c* is not 0 to 255
258	`invalid argument`	The driver refused it: a negative radius, or a centre far off the bitmap
13	`type mismatch`	An argument is a string

SEE ALSO

`SCREEN`, `COLOR`, `PSET`, `LINE`, `PAINT`, `WINDOW`, `POINT`.

EXAMPLE

With a Vera X card. (`SCREEN` clears the console, so this one is shown as it was typed, not as a replayed transcript.)

```
> SCREEN 13 : CIRCLE (100, 100), 20, 200
> PRINT POINT(120, 100); POINT(100, 80); POINT(100, 100)
 200  200  0
> CIRCLE (100, 100), 10, 3, , , , F : PRINT POINT(100, 100); POINT(105, 100)
 3  3
> SCREEN 0
```

CLEAR

A command of the prompt: forgets every variable.

SYNTAX

```
CLEAR
```

DESCRIPTION

Every variable and array of the program is dropped, so that each is 0, empty or not yet made when it is next used. The program's lines stay. A program that was stopped can no longer be continued with `CONT`.

`CLEAR` is one of the commands that work only when typed at the prompt, alone on its line. `RUN` clears the variables itself as it starts.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`CLEAR` in a program's line, or after other statements on a line typed at the prompt

SEE ALSO

`NEW`, `RUN`, `ERASE`, `CONT`.

CLOSE

A statement: closes files.

SYNTAX

```
CLOSE [[#]f [, [#]f] ...]
```

ARGUMENTS

f

A numeric expression: the number of a file that `OPEN` opened.

DESCRIPTION

Each file named is closed, and its number is free to be used by another `OPEN`. With no numbers, every open file is closed.

Closing a number that is not open is not an error: nothing happens. That holds for any number, 0 and numbers above 255 included.

What a program has written with `PRINT #`, `WRITE #` or `PUT` is in the file as each statement finishes, not held back until `CLOSE`; closing only gives up the file.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*f* is a string

NOTES

`RUN` closes every file as it starts, and files are closed when HyBASIC ends.

SEE ALSO

`OPEN`, `FREEFILE`.

CLS

A statement: clears the screen.

SYNTAX

```
CLS
```

DESCRIPTION

Clears the console's text and puts the cursor at the top left. In a graphics mode (`SCREEN` 1 or more) the bitmap is cleared to colour 0 as well, and the last point referenced becomes its middle.

If output is not a console, the bytes that would clear one (an escape sequence, section 13.7) go where output goes.

SEE ALSO

LOCATE, SCREEN, COLOR.

COLOR

A statement: sets the colours of text, or in a graphics mode the colour drawn with.

SYNTAX

```
COLOR [fg] [, bg]
```

ARGUMENTS

fg

A numeric expression: the foreground colour.

bg

A numeric expression: the background colour.

DESCRIPTION

With no graphics mode (SCREEN 0), COLOR sets the colours of the text written from then on. Each is 0 to 15: 0 to 7 are the terminal's eight colours (black, red, green, yellow, blue, magenta, cyan, white), and 8 to 15 are their bright forms. One that is left out is not changed. The colours are set by writing an escape sequence (section 13.7), so what they look like is the terminal's choice.

In a graphics mode (SCREEN 1 or more), *fg* sets the pen: the colour used by the drawing statements when they are given none. *bg* is accepted and ignored.

ERRORS

ERR	MESSAGE	CAUSE
5	illegal function call	A text colour that isn't 0 to 15
13	type mismatch	An argument is a string

SEE ALSO

SCREEN, PALETTE, CLS, PSET, LINE, CIRCLE.

COMMAND$

A string function: the arguments the program was started with, as one string.

SYNTAX

```
COMMAND$
```

RESULT

The arguments after the program's name, joined with one space between each, 255 bytes at most. The empty string if there were none, or if HyBASIC was started without a program.

An argument that held spaces of its own can't be told from two arguments here. `ARG$` gives each one whole.

SEE ALSO

`ARG$`, `ENVIRON$`.

EXAMPLE

```
% echo 'PRINT "["; COMMAND$; "]"' >cmd.bas
% basic cmd.bas one 'two words'
[one two words]
% basic cmd.bas
[]
```

COMPLEX

A numeric function: a complex number made from its two parts.

SYNTAX

```
COMPLEX(re, im)
```

ARGUMENTS

re*, *im

Numeric expressions whose values are real: the real part and the imaginary part.

RESULT

The number *re* + *im* i. The parts keep their kinds: a rational real part with a decimal imaginary part is a complex number of exactly those. If *im* is 0 the result is the real number *re*, not a complex number with nothing in its imaginary part.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*re* or *im* is itself complex
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

NOTES

A complex number can also be written directly, as `1+2i` or `0.5i`: digits with an `i` after them are an imaginary number. The `i` must follow a number. `1+i` is 1 plus the variable `I`.

SEE ALSO

REAL, IMAG, SQR.

EXAMPLE

```
> PRINT COMPLEX(1, 2); COMPLEX(1/2, 0.5); COMPLEX(3, 0); COMPLEX(0, 1) ^ 2
 1+2i  1/2+0.5i  3 -1
> PRINT COMPLEX(1+2i, 3)
illegal function call
```

CONST

A statement: gives names to constant values.

SYNTAX

```
CONST name = value [, name = value] ...
```

ARGUMENTS

name

A name that has not been used: a string's if it ends in `$`, otherwise a number's.

value

A constant expression of the name's type: numbers and strings written out, other constants, and operators and built-in functions applied to them.

DESCRIPTION

The value is worked out when the program is compiled, and the name stands for it from then on. A constant can be used wherever a value can, and it is known in every `SUB` and `FUNCTION` without being `SHARED`.

A constant can't be assigned to, and its name can't be given to anything else.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	*name* is already a constant or a variable; or an assignment to a constant
87	`not a constant`	*value* uses a variable, an array or a `FUNCTION`
13	`type mismatch`	A string for a numeric name, or a number for a string name

NOTES

An integer constant's name is also a symbol in the program's `ASM` blocks, where it stands for its value.

SEE ALSO

DIM, LET, ASM.

EXAMPLE

```
> CONST LIMIT = 5, UNIT$ = "cm", TWICE = LIMIT * 2 : PRINT LIMIT; UNIT$; TWICE
 5 cm 10
> LIMIT = 6
duplicate definition
> V = 3 : CONST LATER = V
not a constant
```

CONT

A command of the prompt: goes on with a program that was stopped.

SYNTAX

```
CONT
```

DESCRIPTION

A program stops, with its state kept, at a `STOP` statement or when CTRL-C is pressed. `CONT` carries on from the statement after the one it stopped at, with every variable as it is now: variables can be looked at, and changed, at the prompt in between.

A program can no longer be continued once it has ended, once one of its lines has been changed, added or removed, or after `NEW`, `LOAD` or `CLEAR`.

ERRORS

ERR	MESSAGE	CAUSE
17	`cannot continue`	No program is stopped
89	`only at the prompt`	`CONT` in a program's line

SEE ALSO

`STOP`, `END`, `RUN`.

EXAMPLE

```
> 10 PRINT "one" : STOP : PRINT "two"
> 20 PRINT "three"
> RUN
one
Break in line 10
> CONT
two
three
> CONT
cannot continue
```

COS

A numeric function: the cosine of a real number.

SYNTAX

```
COS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an angle in radians.

RESULT

The cosine of *x*, from −1 to 1. `COS(0)` is the integer 1. Any other result is a decimal, correct to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

An angle in degrees is turned into radians by multiplying by `PI / 180`.

`PI` is itself rounded to the digits set, so `COS(PI / 2)` is not exactly 0. It is a very small number: the cosine of the angle that `PI / 2` really is.

SEE ALSO

`SIN`, `TAN`, `ATN`, `PI`, `DIGITS`.

EXAMPLE

```
> PRINT COS(0); COS(1); COS(PI); COS(60 * PI / 180)
 1  0.540302305868 -1  0.5
```

CSRLIN

A numeric function: the row the cursor is on.

SYNTAX

```
CSRLIN
```

RESULT

The cursor's row, counted from 1 at the top of the screen. 0 if HyBASIC has no console (its input or output is a file or a pipe).

DESCRIPTION

HyBASIC does not keep track of the row. It asks the terminal each time, by writing a cursor position request (section 13.7) and reading the terminal's reply, so the answer is right whatever has been written to the screen, by this program or another.

NOTES

The column is `POS(0)`.

SEE ALSO

`POS`, `LOCATE`.

CVN

A numeric function: the number that a string of bytes holds, in the stored format.

SYNTAX

```
CVN(s$)
```

ARGUMENTS

s$

A string expression: a number's bytes, as `MKN$` makes them and as `PUT` writes a numeric variable to a file.

RESULT

The number. It is exactly the number that was stored, of the same kind: `CVN(MKN$(x))` is *x* for every number *x*.

DESCRIPTION

A number's stored form is as long as the number needs: a small integer takes one byte, a rational or a long integer more.

It replaces QuickBASIC's `CVI`, `CVL`, `CVS` and `CVD`, as `MKN$` replaces `MKI$` and the rest: there is one kind of number, so one pair of functions.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is not a number's stored form
13	`type mismatch`	*s$* is a number

NOTES

A string that was never a number may still be some number's stored form, and then there is no error: `CVN("x")` is –8.

SEE ALSO

MKN$, GET, PUT.

EXAMPLE

```
> PRINT CVN(MKN$(1/3)); CVN(MKN$(-7)); CVN(MKN$(2+3i)); LEN(MKN$(7)); LEN(MKN$(1/3))
 1/3 -7  2+3i  1  3
```

DATA

A statement: holds values in the program for READ to take.

SYNTAX

```
DATA item [, item] ...
```

ARGUMENTS

item

A value, written out. It is not an expression: nothing in it is evaluated.

DESCRIPTION

DATA does nothing when control passes over it. Its items are collected when the program is compiled: all the DATA statements of the program, in the order of its lines, make one list, and READ takes the next item from that list each time it fills a variable. RESTORE moves the place back.

An item is the text between two commas, with the spaces before and after it dropped. Text in double quotes is taken as it stands, commas and spaces included, without the quotes. An empty item, two commas with nothing between them, is allowed.

What an item means is decided by the variable that reads it. Read into a string variable, it is its text. Read into a numeric variable, it is read as a number in any form a program's numbers may take (12, -7/2, 2.5, &HFF); text that isn't a number, and an empty item, give 0.

ERRORS

ERR	MESSAGE	CAUSE
4	out of DATA	A READ when every item has been taken. The error is READ's, at its line

NOTES

Because the items aren't expressions, DATA 2 + 2 holds the five characters 2 + 2: nothing adds them up.

A DATA statement can be anywhere in the program, before or after the READ that uses it.

SEE ALSO

READ, RESTORE.

EXAMPLE

```
> 10 DATA 12, hello there , "a, b", 1/2, , &HFF
> 20 READ A, B$, C$, D, E$, F
> 30 PRINT A; "["; B$; "]["; C$; "]"; D; "["; E$; "]"; F
> 40 READ G
> RUN
 12 [hello there][a, b] 1/2 [] 255
line 40: out of DATA
```

DATE$

A string function: today's date.

SYNTAX

```
DATE$
```

RESULT

A string of ten characters, `yyyy-mm-dd`: the year, the month and the day of the system's clock. On a Hydra with no clock chip, the clock starts at 2000-01-01 when the system starts.

ERRORS

ERR	MESSAGE	CAUSE
73	`not done yet`	An assignment to `DATE$`. QuickBASIC sets the date that way; HyBASIC doesn't set the clock

NOTES

The form is not QuickBASIC's `mm-dd-yyyy`. Strings in this form sort in date order.

SEE ALSO

`TIME$`, `TIMER`.

DECLARE

A statement: accepted, and ignored.

SYNTAX

```
DECLARE SUB name [(parameters)]
DECLARE FUNCTION name [(parameters)]
```

DESCRIPTION

QuickBASIC needs a `DECLARE` for a procedure that is used before it is defined. HyBASIC compiles the whole program before it runs any of it, and finds every `SUB` and `FUNCTION` first, so a declaration tells it nothing. The statement is read past without being checked: its parameter list need not match the procedure's.

It is here so that programs written for QuickBASIC load unchanged.

SEE ALSO

`SUB`, `FUNCTION`.

DEFINT, DEFLNG, DEFSNG, DEFDBL, DEFSTR

Statements: set the type of names by their first letter.

SYNTAX

```
DEFSTR letters [, letters] ...
DEFINT letters [, letters] ...
```

ARGUMENTS

letters

One letter, or a range of letters with a hyphen between: `S`, `A-C`.

DESCRIPTION

A name normally belongs to a string if it ends in `$` and to a number if it doesn't. After `DEFSTR`, a name with no `$` whose first letter is one of those given is a string's too.

`DEFINT`, `DEFLNG`, `DEFSNG` and `DEFDBL` all do the opposite, and are the same statement: they make names that start with those letters numbers' again. (In QuickBASIC they choose between four kinds of number. HyBASIC has one.)

The statement takes effect where it stands in the program's text, for the lines after it, not when control reaches it.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	A number assigned to a name that `DEFSTR` has made a string's, or the reverse

SEE ALSO

`DIM`, `OPTION`.

EXAMPLE

```
> 10 DEFSTR S-T : S = "str" : T1 = "t" : PRINT S; T1
> 20 DEFINT S : S2 = 5 : PRINT S2
> RUN
strt
 5
```

DELETE

A command of the prompt: removes lines from the program.

SYNTAX

```
DELETE from[-to]
```

ARGUMENTS

from, to
: Line numbers.

DESCRIPTION

The lines numbered *from* to *to*, both included, are removed. With *from* alone, that one line is. Numbers in the range that no line has are passed over, so a range can name lines that aren't there, and a `DELETE` that finds none is not an error.

A program that was stopped can't be continued after a `DELETE`.

Typing a line number with nothing after it removes that one line, which is usually quicker.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	No line number; or a range with no *from*
89	`only at the prompt`	`DELETE` in a program's line

SEE ALSO

`LIST`, `NEW`, `EDIT`.

EXAMPLE

```
> 10 PRINT 1
> 20 PRINT 2
> 30 PRINT 3
> 40 PRINT 4
> DELETE 20-30
> LIST
10 PRINT 1
40 PRINT 4
```

DENOMINATOR

A numeric function: the denominator of a real number written as a fraction in lowest terms.

SYNTAX

```
DENOMINATOR(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

A positive integer. A rational gives its denominator, with any common factor already taken out: 6/8 is kept as 3/4, so it gives 4. An integer gives 1. A decimal is turned into the fraction it exactly equals first: 0.75 is 3/4, and gives 4.

`NUMERATOR(x) / DENOMINATOR(x)` is *x* for every real *x*; the sign is the numerator's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`NUMERATOR`, `RATIONAL`, `FIXED`, `GCD`.

EXAMPLE

```
> PRINT NUMERATOR(-6/8); DENOMINATOR(-6/8); DENOMINATOR(7); NUMERATOR(0.75);
DENOMINATOR(0.75)
-3  4  1  3  4
```

DIGITS

A statement and a function: sets, or reports, how many significant digits the mathematical functions work to.

SYNTAX

```
DIGITS n
DIGITS
```

ARGUMENTS

n

A numeric expression from 1 to 100, rounded to the nearest integer.

RESULT

`DIGITS`, used as a function, is the current setting: 12 when HyBASIC starts.

DESCRIPTION

Addition, subtraction, multiplication and division of integers, rationals and decimals are exact, and `DIGITS` has nothing to do with them. It governs the functions whose results generally can't be written exactly: `SQR`, `EXP`, `LOG`, `SIN`, `COS`, `TAN`, `ATN`, `PI` and a power with a fractional exponent. Each of those

returns a decimal rounded to *n* significant digits. `RND` returns a decimal of *n* digits too.

A result that happens to be exact comes back exact whatever the setting: `SQR(16)` is the integer 4.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 100
13	`type mismatch`	*n* is a string

NOTES

A value that was worked out at one setting keeps the digits it has when the setting changes. To cut a number to some number of decimal places, use `FIXED`.

SEE ALSO

`BASE`, `FIXED`, `PI`, `SQR`, `RND`.

EXAMPLE

```
> PRINT DIGITS; SQR(2); 1/3
 12  1.41421356237  1/3
> DIGITS 5 : PRINT SQR(2); PI; 1/3 : DIGITS 12
 1.4142  3.1416  1/3
> DIGITS 0
illegal function call
```

DIM

A statement: makes arrays, and declares variables with a type.

SYNTAX

```
DIM [SHARED] name[(bounds [, bounds] ...)] [AS type] [, name ...] ...
```

where *bounds* is

```
upper
lower TO upper
```

ARGUMENTS

name

The array's or the variable's name.

lower, upper

Numeric expressions, each rounded to the nearest integer: the lowest and the highest index in that dimension. If *lower* is left out it is 0, or 1 after `OPTION BASE 1`. An array can have from 1 to 8 dimensions.

type

INTEGER, LONG, SINGLE, DOUBLE or NUMBER, which all mean a number; STRING; or the name of a TYPE, for a record. Without AS, the type is the name's own: a string if it ends in $.

DESCRIPTION

An array is made when the statement runs, so its bounds can be worked out by the program. Every element starts as 0, or as the empty string. `DIM A(3)` has the four elements `A(0)` to `A(3)`.

An array that is used without having been made is made on the spot, with indexes 0 to 10 in each dimension it is used with (from 1 after `OPTION BASE 1`). DIM is needed only for other sizes.

A variable with no bounds is declared with its type: `DIM Count AS INTEGER`, `DIM P AS Point`. A numeric type is a statement of intent only. Every numeric variable holds any number, so a variable declared INTEGER can be given 1/3 and keeps it.

SHARED makes the names known inside every SUB and FUNCTION as well as in the main program: see SHARED.

ERRORS

ERR	MESSAGE	CAUSE
10	`duplicate definition`	The array has been made already. ERASE it first, or use REDIM
9	`subscript out of range`	A negative *upper*; *lower* above *upper*; more than 8 dimensions. Also: an index outside an array's bounds, wherever the array is used
7	`out of memory`	No room for the array
13	`type mismatch`	A bound that is a string

NOTES

A name followed by parentheses is taken to be an array's unless it is a FUNCTION's. A misspelled function name is therefore not an error: it is a new array, whose elements are all 0.

SEE ALSO

REDIM, ERASE, OPTION, LBOUND, UBOUND, TYPE, SHARED, STATIC.

EXAMPLE

```
> DIM A(3) : A(3) = 7 : PRINT A(0); A(3); LBOUND(A); UBOUND(A)
 0  7  0  3
> A(4) = 1
subscript out of range
> DIM A(5)
duplicate definition
> DIM C(1 TO 3, -2 TO 2) : C(3, -2) = 9 : PRINT C(3, -2); LBOUND(C, 2); UBOUND(C, 2)
 9 -2  2
> B(10) = 1 : PRINT UBOUND(B)
 10
> DIM Q AS INTEGER : Q = 1/3 : PRINT Q
 1/3
```

DIR$

A string function: the names in a directory, one at a time.

SYNTAX

```
DIR$(path$)
DIR$
```

ARGUMENTS

path$

A string expression: a directory's name.

RESULT

`DIR$(path$)` is the first name in the directory, and each `DIR$` after it, with no argument, is the next. A name that is itself a directory has a `/` after it. The empty string when there are no more, and for every call after that. The names come in the directory's own order, which is not alphabetical.

The empty string, too, if *path$* is a file and not a directory.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name as *path$*
13	`type mismatch`	*path$* is a number

NOTES

path$ names a directory. It is not a pattern: QuickBASIC's `DIR$("*.BAS")` has no meaning here. To pick out some of the names, test each one as it comes.

SEE ALSO

`FILES`, `CHDIR`, `OPEN`.

EXAMPLE

```
> CHDIR "/ram" : MKDIR "sub" : OPEN "a.txt" FOR OUTPUT AS #1 : CLOSE
> N$ = DIR$("/ram") : DO WHILE N$ <> "" : PRINT N$; " "; : N$ = DIR$ : LOOP : PRINT
bin/ lib/ sub/ a.txt
```

DO ... LOOP

A statement block: a loop that repeats while, or until, a condition holds.

SYNTAX

```
DO [{WHILE | UNTIL} condition]
    [statements]
LOOP

DO
    [statements]
LOOP [{WHILE | UNTIL} condition]
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0.

DESCRIPTION

With the condition after `DO`, it is tested before each pass, the first included, so the statements may not run at all. With the condition after `LOOP`, it is tested after each pass, so the statements run at least once.

`WHILE` goes on for as long as the condition holds. `UNTIL` goes on for as long as it does not.

With no condition in either place the loop repeats for ever, until `EXIT DO` leaves it, an error stops it, or CTRL-C is pressed. `EXIT DO` leaves the innermost `DO` loop at once, whatever its condition, and goes on after its `LOOP`.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	A `DO` with no `LOOP`, or a `LOOP` with no `DO`; a condition in both places; the loop and another block overlapping. Found when the program is compiled
13	`type mismatch`	The condition is a string

NOTES

At the prompt a loop must be whole on its line, since each line typed is compiled alone.

`WHILE` ... `WEND` is the same as `DO WHILE` ... `LOOP`.

SEE ALSO

`EXIT`, `WHILE`, `FOR`.

EXAMPLE

```
> I = 0 : DO : I = I + 1 : LOOP UNTIL I = 3 : PRINT I
 3
> I = 5 : DO WHILE I < 3 : I = I + 1 : LOOP : PRINT I
 5
> I = 5 : DO : I = I + 1 : LOOP WHILE I < 3 : PRINT I
 6
```

EDIT

A command of the prompt: opens the program in the screen editor.

SYNTAX

```
EDIT [line]
```

ARGUMENTS

line

A line number or a label: where the editor's cursor starts.

DESCRIPTION

The program's text is written to a file in `/ram`, and the screen editor, `edit` (Chapter 11), is started on it with the cursor at *line*. With no *line*, the cursor starts at the line of the last error, if there has been one.

When the editor is left, the file is read back as the program and compiled, and the first error found, if there is one, is reported with its line. `EDIT` again goes to it.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`EDIT` in a program's line

SEE ALSO

`LIST`, `LOAD`, `SAVE`, `DELETE`.

ELSE, ELSEIF

Parts of `IF`: `ELSE` begins the statements for a condition that didn't hold, and `ELSEIF` tests another. `CASE ELSE` is the last choice of a `SELECT CASE`. See `IF` and `SELECT`.

END

A statement: ends the program. Also the first word of the lines that close a block.

SYNTAX

```
END
END {IF | SELECT | SUB | FUNCTION | TYPE | ASM}
```

DESCRIPTION

`END` alone stops the program and returns to the prompt, or, when HyBASIC was started with a program to run, ends HyBASIC. Nothing is printed. A program also ends when control runs off its last line, so `END` is needed only to stop sooner, typically before the lines of a subroutine that `GOSUB` reaches.

A program that has ended can't be continued with `CONT`. To stop in a way that can be, use `STOP`.

The two-word forms each close the block of that name: see `IF`, `SELECT`, `SUB`, `FUNCTION`, `TYPE` and `ASM`.

SEE ALSO

`STOP`, `SYSTEM`, `CONT`.

EXAMPLE

```
> 10 PRINT "a" : END : PRINT "b"
> RUN
a
> CONT
cannot continue
```

ENV$, ENVIRON$

String functions: the value of a variable of the environment.

SYNTAX

```
ENVIRON$(name$)
ENV$(name$)
```

ARGUMENTS

name$

A string expression: the variable's name, in the case it has. The shell's own variables are in small letters.

RESULT

The variable's value. A value is a list of words (section 8.4); they are returned joined with one space between each. The empty string if there is no such variable.

The two names are the same function.

DESCRIPTION

The environment is the set of named values a task starts with, copied from the task that started it: for HyBASIC, usually a shell, whose variables they are: `path`, `prompt` and the rest. Each is also a file in `/env` (section 17.7).

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*name$* is a number

SEE ALSO

`ENVIRON`, `ARG$`, `COMMAND$`, `SHELL`.

EXAMPLE

```
> PRINT ENVIRON$("path"); "|"; ENV$("nosuch"); "|"
. /bin||
```

ENVIRON

A statement: sets, or removes, a variable of the environment.

SYNTAX

```
ENVIRON "name=value"
```

ARGUMENTS

A string expression: the variable's name, an equals sign, and its value.

DESCRIPTION

The variable is set in HyBASIC's own environment. The value is one word, whatever spaces are in it. Programs that HyBASIC starts afterwards with `SHELL` get a copy of the environment and see the variable; the shell that started HyBASIC does not, since a task can't change its parent's environment.

With nothing after the equals sign, the variable is removed.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	No equals sign in the string
13	`type mismatch`	The argument is a number

SEE ALSO

`ENVIRON$`, `SHELL`.

EXAMPLE

```
> ENVIRON "colour=red" : PRINT ENVIRON$("colour") : SHELL "cat /env/colour; echo"
red
red
> ENVIRON "colour=" : PRINT "["; ENVIRON$("colour"); "]"
[]
```

EOF

A numeric function: whether a file has been read to its end.

SYNTAX

```
EOF(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file.

RESULT

−1 (true) if there is nothing more to read from file *f*, and 0 (false) if there is. For a file opened `FOR OUTPUT` or `FOR APPEND`, always −1.

DESCRIPTION

The usual loop reads while there is something to read: `DO UNTIL EOF(1) : LINE INPUT #1, L$ : ... : LOOP`. Testing first matters, because reading at the end is an error (62, `input past end of file`), and an empty file is at its end from the start.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
13	`type mismatch`	*f* is a string

SEE ALSO

`OPEN`, `INPUT`, `LINE INPUT`, `LOF`, `LOC`.

EXAMPLE

```
> OPEN "/ram/two.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT #1, "two" : CLOSE
> OPEN "/ram/two.txt" FOR INPUT AS #1
> DO UNTIL EOF(1) : LINE INPUT #1, L$ : PRINT L$; EOF(1) : LOOP : CLOSE
one 0
two-1
```

EQV

An operator: bitwise equivalence.

SYNTAX

```
a EQV b
```

RESULT

An integer. Each bit of it is 1 where the same bit of *a* and of *b* are alike (both 0 or both 1), and 0 where they differ. It is `NOT (a XOR b)`.

For two truth values, −1 and 0, the result is true when they are the same.

The operands are evaluated, rounded and treated as `AND`'s are: see `AND`. `EQV` binds less tightly than `AND`, `OR` and `XOR`, and more tightly than `IMP`.

EXAMPLE

```
> PRINT 5 EQV 3; (1 = 1) EQV (2 = 3); 0 EQV 0
-7  0 -1
```

ERASE

A statement: drops arrays.

SYNTAX

```
ERASE array [, array] ...
```

ARGUMENTS

array

An array's name, with no parentheses.

DESCRIPTION

Each array named is dropped, and the memory it took is free again. The name is then as if it had never been used: a `DIM` can make it again with other bounds, and using it without one makes it afresh with indexes 0 to 10, every element 0 or empty.

A name that isn't an array's is passed over without an error.

SEE ALSO

`DIM`, `REDIM`, `CLEAR`.

EXAMPLE

```
> DIM A(3) : A(3) = 7 : ERASE A : DIM A(50) : PRINT A(3); UBOUND(A)
 0  50
```

ERL, ERR, ERR$

Functions: the line, the number and the message of the last error.

SYNTAX

```
ERR
ERR$
ERL
```

RESULT

`ERR` is the number of the most recent error, and 0 if there has been none. The numbers are QuickBASIC's where it has an error of the same meaning (5 is `illegal function call`, 53 is `file not found`); the Hydra's own start at 80. An error reported by the system that has no number of HyBASIC's is 256 plus the system's code (Appendix G).

`ERR$` is that error's message, as it is printed: in small letters, with no line number. `error 0` if there has been none.

`ERL` is the number of the line the error was on. If that line has no number, it is the number of the nearest numbered line before it; in a program with no line numbers at all, it is the line's position in its file, counted from 1. 0 if the error was in a line typed at the prompt, or if there has been none.

DESCRIPTION

The three keep their values until the next error: a statement that succeeds doesn't clear them. They are meant to be read in an error handler (`ON ERROR GOTO`), where they tell it what happened and where.

SEE ALSO

`ON`, `RESUME`, `ERROR`.

EXAMPLE

```
> PRINT ERR; ERL; ERR$
 0  0 error 0
> 10 X = 1 / 0
> RUN
line 10: division by zero
> PRINT ERR; ERL; ERR$
 11  10 division by zero
```

ERROR

A statement: raises an error.

SYNTAX

```
ERROR n
```

ARGUMENTS

n

A numeric expression from 1 to 255: the error's number.

DESCRIPTION

The program behaves exactly as if error *n* had happened at this statement. If an `ON ERROR GOTO` handler is in force, control goes to it, with `ERR` equal to *n* and `ERL` this line; otherwise the program stops and the error's message is printed.

If *n* is one of HyBASIC's error numbers, the message is that error's. Any other number from 1 to 255 can be used for an error of the program's own; its message is `error` and the number.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is less than 1 or more than 255

SEE ALSO

ON, RESUME, ERR.

EXAMPLE

```
> 10 ON ERROR GOTO 50
> 20 ERROR 200
> 30 PRINT "after"
> 40 END
> 50 PRINT "caught"; ERR; ERL; ERR$ : RESUME NEXT
> RUN
caught 200  20 error 200
after
```

EXIT

A statement: leaves a loop or a procedure early.

SYNTAX

```
EXIT {FOR | DO | SUB | FUNCTION}
```

DESCRIPTION

`EXIT FOR` and `EXIT DO` leave the innermost loop of that kind that the statement is inside, and the program goes on with the statement after the loop's `NEXT` or `LOOP`. A `FOR` loop's variable keeps the value it has. An `EXIT DO` inside a `FOR` loop that is itself inside a `DO` loop leaves both.

`EXIT SUB` and `EXIT FUNCTION` return from the procedure at once. A function returns the value last assigned to its name, or 0 or the empty string if none has been.

ERRORS

ERR	MESSAGE	CAUSE
80	`block without its start or end`	`EXIT FOR` or `EXIT DO` that is not inside such a loop
85	`not in a procedure`	`EXIT SUB` or `EXIT FUNCTION` outside a `SUB` or a `FUNCTION`

NOTES

There is no `EXIT WHILE`, and `EXIT DO` doesn't leave a `WHILE` … `WEND` loop. A loop that needs an early way out should be written with `DO`.

SEE ALSO

FOR, DO, SUB, FUNCTION.

EXAMPLE

```
> 10 FOR I = 1 TO 100 : IF I * I > 50 THEN EXIT FOR
> 20 NEXT : PRINT I
> RUN
 8
```

EXP

A numeric function: e raised to a power.

SYNTAX

```
EXP(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

e to the power *x*, where e is 2.71828... `EXP(0)` is the integer 1. Any other result is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

There is no overflow. A large result is a decimal of that many significant digits followed by zeros, and is printed in full.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`LOG`, `SQR`, `DIGITS`.

EXAMPLE

```
> PRINT EXP(0); EXP(1); EXP(-1); EXP(1/2)
 1  2.71828182846  0.367879441171  1.6487212707
> PRINT EXP(100)
 2688117141820000000000000000000000000000000000
```

FIB

A numeric function: a Fibonacci number.

SYNTAX

```
FIB(n)
```

ARGUMENTS

n

A numeric expression whose value is an integer, 0 or more.

RESULT

The *n*th Fibonacci number, an exact integer of whatever length: `FIB(0)` is 0, `FIB(1)` is 1, and each one after is the sum of the two before it.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative, or is not an integer
13	`type mismatch`	*n* is a string

SEE ALSO

`GCD`.

EXAMPLE

```
> PRINT FIB(0); FIB(1); FIB(2); FIB(30); FIB(100)
 0  1  1  832040  354224848179261915075
```

FILES

A statement: lists the names in a directory.

SYNTAX

```
FILES [path$]
```

ARGUMENTS

path$

A string expression: a directory's name. The current directory if it is left out.

DESCRIPTION

The names are printed across the screen, each in a column 16 characters wide, five to a line, in the directory's own order. A name that is a directory has a `/` after it. Nothing else is shown: for sizes and dates, use `SHELL "ls -l"`.

If *path$* is a file and not a directory, nothing is listed.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such name as *path$*
13	`type mismatch`	*path$* is a number

NOTES

path$ is a directory, not a pattern: `FILES "*.BAS"` looks for a directory of that name.

SEE ALSO

`DIR$`, `CHDIR`, `KILL`, `NAME`, `SHELL`.

EXAMPLE

```
> FILES "/rom/lib"
as/             basic/          edit/           font/           forth/
hylang/         namespace       profile         shell           windows
```

FIX

A numeric function: a real number with its fraction removed.

SYNTAX

```
FIX(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The integer part of *x*: *x* rounded toward zero. For a positive number this is `INT(x)`. For a negative number that isn't whole it is one more: `FIX(-2.5)` is −2, where `INT(-2.5)` is −3. The result is an exact integer of whatever length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

SEE ALSO

`INT`, `CINT`, `FIXED`.

EXAMPLE

```
> PRINT FIX(2.5); FIX(-2.5); FIX(7/2); FIX(-7/2); INT(-2.5)
 2 -2  3 -3 -3
```

FIXED

A numeric function: a real number as a decimal with a given number of places.

SYNTAX

```
FIXED(x, n)
```

ARGUMENTS

x

A numeric expression whose value is real.

n

A numeric expression, 0 or more: the number of decimal places.

RESULT

A decimal: *x* with the digits after its *n*th decimal place cut off. The digits are cut, not rounded, so the result is never further from zero than *x*: `FIXED(2/3, 2)` is 0.66, and `FIXED(-2/3, 2)` is −0.66. Zeros at the end are not kept, so `FIXED(5, 2)` is 5.

DESCRIPTION

This is the way to turn a rational into a decimal. `1/3` is kept and printed as `1/3`; `FIXED(1/3, 6)` is 0.333333. *n* is not limited by `DIGITS`, which is about the mathematical functions.

To round instead of cutting, add half of the last place first: `FIXED(x + 0.005, 2)` rounds a positive *x* to two places.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	An argument is a string

SEE ALSO

`RATIONAL`, `FIX`, `CINT`, `DIGITS`, `PRINT` (`USING`).

EXAMPLE

```
> PRINT FIXED(22/7, 4); FIXED(2/3, 2); FIXED(-2/3, 2); FIXED(5, 2); FIXED(1.239, 2)
 3.1428  0.66 -0.66  5  1.23
> PRINT FIXED(1/3, 30)
 0.333333333333333333333333333333
```

FOR ... NEXT

A statement block: a loop that counts a variable from one value to another.

SYNTAX

```
FOR v = start TO limit [STEP step]
    [statements]
NEXT [v [, v] ...]
```

ARGUMENTS

v

The loop's counter: a simple numeric variable. An array's element can't be one.

start, limit, step

Numeric expressions. *step* is 1 if `STEP` is left out.

DESCRIPTION

1. *start* is assigned to *v*, and *limit* and *step* are evaluated and kept. They are evaluated this once only: a variable they were worked out from can change inside the loop without changing them.
2. The test: if *step* is zero or more and *v* is greater than *limit*, or *step* is negative and *v* is less than *limit*, the loop ends, and the program goes on after its `NEXT`. The test is made before the first pass too, so the statements may not run at all; *v* then keeps the value *start*.
3. Otherwise the statements run.
4. At `NEXT`, *step* is added to *v*, and the loop goes back to the test at step 2.

When the loop ends at the test, *v* holds the first value that failed it: after `FOR I = 1 TO 3`, `I` is 4.

The addition is exact. A loop with a fractional step, `STEP 0.1` or `STEP 1/3`, reaches its limit on the last pass, where a binary floating-point BASIC may stop one short.

The statements may assign to *v*; `NEXT` adds *step* to whatever it then holds.

`EXIT FOR` leaves the innermost loop at once, with *v* as it is.

`NEXT` with no variable closes the innermost open loop. `NEXT v` does the same, and the compiler checks that *v* is that loop's counter. `NEXT j, i` is `NEXT j : NEXT i`.

ERRORS

ERR	MESSAGE	CAUSE
26	`FOR without NEXT`	A `FOR` whose `NEXT` isn't there. Found when the program is compiled: nothing has run
1	`NEXT without FOR`	A `NEXT` with no loop open, or naming a variable that isn't the innermost loop's counter
80	`block without its start or end`	The loop and another block overlap: a `NEXT` inside a one-line `IF`, say
13	`type mismatch`	*v* is a string variable, or *start*, *limit* or *step* a string
2	`syntax error`	*v* is not a simple variable

NOTES

With a *step* of 0 the counter never changes and the loop doesn't end by itself: only `EXIT FOR`, an error or CTRL-C ends it.

At the prompt, a line is compiled alone, so a loop typed there must be whole on its line: `FOR I = 1 TO 3` by itself is `FOR without NEXT`.

SEE ALSO

`EXIT`, `DO...LOOP`, `WHILE...WEND`.

EXAMPLE

```
> FOR I = 1 TO 3 : PRINT I; : NEXT : PRINT "then"; I
 1  2  3 then 4
> FOR I = 5 TO 1 : PRINT I; : NEXT : PRINT "then"; I
then 5
> FOR X = 0 TO 1 STEP 1/3 : PRINT X; : NEXT : PRINT
 0  1/3  2/3  1
> N = 3 : FOR I = 1 TO N : N = 10 : PRINT I; : NEXT : PRINT
 1  2  3
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : PRINT I; J; : NEXT J, I : PRINT
 1  1  1  2  2  1  2  2
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : NEXT I
NEXT without FOR
```

FRE

A numeric function: how much memory is free.

SYNTAX

```
FRE()
FRE(x)
```

ARGUMENTS

x

Any expression. It is evaluated and ignored; QuickBASIC's forms `FRE(0)` and `FRE("")` both work.

RESULT

The number of bytes left for the program's data: its strings, its arrays and its numbers too long for a variable's own five bytes. The figure is the room in the RAM banks HyBASIC has not yet used, so it falls as arrays are made and strings grow, and rises when they are dropped.

NOTES

With three RAM modules a new HyBASIC has about 300K free.

SEE ALSO

`DIM`, `ERASE`, `CLEAR`.

EXAMPLE

The figure itself depends on the machine. An array of 5,001 numbers takes five bytes for each, and twelve for the array:

```
> A = FRE() : DIM BIG(5000) : PRINT A - FRE()
 25017
```

FREEFILE

A numeric function: a file number that is not in use.

SYNTAX

```
FREEFILE
```

RESULT

The lowest number, from 1 up, that no open file has. It is the number to give the next `OPEN` in a routine that can't know which numbers the rest of the program is using.

NOTES

`FREEFILE` finds a number. It doesn't reserve it: call it again only after the `OPEN`, or it returns the same one.

Since 8 files can be open at once and numbers go to 255, there is always a free number. When 8 are open the `OPEN` fails instead, with `too many files`.

SEE ALSO

`OPEN`, `CLOSE`.

EXAMPLE

```
> PRINT FREEFILE : OPEN "/ram/x" FOR OUTPUT AS #1 : OPEN "/ram/y" FOR OUTPUT AS #3
 1
> PRINT FREEFILE : CLOSE #1 : PRINT FREEFILE : CLOSE
 2
 1
```

FUNCTION ... END FUNCTION

A block: defines a function, which returns a value and is used in expressions.

SYNTAX

```
FUNCTION name [(parameter [, parameter] ...)] [STATIC]
    [statements]
    name = value
END FUNCTION
```

ARGUMENTS

name

The function's name. It returns a string if the name ends in `$`, and a number otherwise.

parameter

A variable's name, optionally followed by `AS type`; or an array's, written with empty parentheses.

STATIC

The function's variables keep their values from one call to the next. See `STATIC`.

DESCRIPTION

The function is called by using its name in an expression, with an argument for each parameter: `PRINT Area(3, 4)`. A function with no parameters is used as a bare name.

The value returned is whatever was last assigned to the function's name when the function ends, at `END FUNCTION` or at an `EXIT FUNCTION`. If nothing was assigned, it is 0, or the empty string.

Arguments are passed as a `SUB`'s are (see `CALL`): a variable or an array's element by reference, so that assigning to the parameter changes it; any other expression by value. To protect a variable, pass it in parentheses of its own.

The variables used inside the function are its own. A name there is a new variable, 0 or empty at each call, even if the main program has a variable of the same name; the main program's is not touched. Names declared `SHARED`, and constants, are the exceptions.

A function may call itself. Each call has its own parameters and variables.

Functions are found when the program is compiled, wherever they are in its text, so one can be used on a line before its definition. When control reaches a `FUNCTION` line in the ordinary run of the program, it skips to after `END FUNCTION`.

ERRORS

ERR	MESSAGE	CAUSE
37	`argument-count mismatch`	A call with more arguments, or fewer, than the function has parameters
10	`duplicate definition`	Two procedures of one name
13	`type mismatch`	An argument of the wrong type; a string assigned to a numeric function's name
80	`block without its start or end`	`FUNCTION` with no `END FUNCTION`
2	`syntax error`	`FUNCTION` typed at the prompt: a function can be defined only in a program

NOTES

A name followed by parentheses that is not a function's is an array's. Calling a function that doesn't exist, or misspelling one, is therefore not an error: it makes an array of that name and returns its element, 0.

SEE ALSO

`SUB`, `CALL`, `EXIT`, `SHARED`, `STATIC`, `DECLARE`.

EXAMPLE

```
> 10 FUNCTION Fact (N)
> 20 IF N <= 1 THEN Fact = 1 ELSE Fact = N * Fact(N - 1)
> 30 END FUNCTION
> 40 FUNCTION Bump (V) : V = V + 1 : X = 99 : Bump = V : END FUNCTION
> 50 PRINT Fact(5); Fact(25)
> 60 X = 1 : Z = 1 : PRINT Bump(Z); Z; Bump((Z)); Z; X
> 70 PRINT Fcat(5)
> RUN
 120  15511210043330985984000000
 2  2  3  2  1
 0
```

GCD

A numeric function: the greatest common divisor of two integers.

SYNTAX

```
GCD(a, b)
```

ARGUMENTS

a, b

Numeric expressions whose values are real. Each is rounded to the nearest integer, a half going to the even one.

RESULT

The largest integer that divides both *a* and *b* exactly. It is never negative: the signs of *a* and *b* make no difference. `GCD(a, 0)` is the size of *a*, and `GCD(0, 0)` is 0. The integers can be of any length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	An argument is complex
13	`type mismatch`	An argument is a string
2	`syntax error`	One argument only

NOTES

The least common multiple of two positive integers is `a * b / GCD(a, b)`.

Rationals are always kept in lowest terms, so there is no need for `GCD` to reduce a fraction.

SEE ALSO

`MOD`, `NUMERATOR`, `DENOMINATOR`, `FIB`.

EXAMPLE

```
> PRINT GCD(12, 18); GCD(-12, 18); GCD(0, 5); GCD(17, 5); GCD(2 ^ 64, 2 ^ 40)
 6  6  5  1  1099511627776
```

GET

A statement: reads a value from a `BINARY` file, at a given position.

SYNTAX

```
GET [#]f, [position], v
```

ARGUMENTS

f

A numeric expression: the number of a file opened `FOR BINARY`.

position

A numeric expression: where to read, counted in bytes from 1 for the file's first byte. If it is left out (the two commas are still written) or is less than 1, the read starts where the last `GET` or `PUT` on the file ended.

v

A variable: where the value goes. Its type and, for a string, its present length decide how many bytes are read.

DESCRIPTION

A string variable is filled with as many bytes from the file as it holds now. To read ten bytes, make the variable ten bytes long first, with `SPACE$(10)` say. If the file ends first, the bytes that weren't there are left as zero bytes; the variable keeps its length.

A numeric variable reads one number in the stored format, the form `PUT` writes and `MKN$` makes. A number takes as many bytes as it needs, which the number's own first bytes say, so numbers written one after another are read back one after another without their lengths being known.

After the read, the file's position is just past what was read.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* was not opened `FOR BINARY`
62	`input past end of file`	A number was to be read, and the file had ended

NOTES

There are no records and no `FIELD`: QuickBASIC's random files are not here. `GET` and `PUT` always work in bytes, and a program that wants records of a fixed size works out each one's position itself.

SEE ALSO

`PUT`, `OPEN`, `SEEK`, `LOC`, `LOF`, `CVN`, `MKN$`, `INPUT$`.

EXAMPLE

```
> OPEN "/ram/b.dat" FOR BINARY AS #1 : A$ = "HELLO" : PUT #1, 1, A$ : N = 1/3 : PUT #1,
, N
> PRINT LOF(1) : B$ = SPACE$(3) : GET #1, 2, B$ : PRINT B$
 8
ELL
> GET #1, 6, M : PRINT M; EOF(1) : CLOSE
 1/3 -1
```

GOSUB

A statement: calls a subroutine, which is a part of the program that ends with `RETURN`.

SYNTAX

```
GOSUB {line | label}
```

ARGUMENTS

line

A line number.

label

A label: a name with a colon after it at the start of a line that has no number.

DESCRIPTION

Control goes to the line named, and the place to come back to is remembered. The next `RETURN` that runs comes back to the statement after the `GOSUB`.

Subroutines can call others, and themselves; each `RETURN` goes back to the most recent `GOSUB` that hasn't been returned from.

A subroutine is not a block: it is only lines of the program, and nothing stops control from running into them from above. The usual arrangement is to put the subroutines at the end of the program, with an `END` before them.

A subroutine shares every variable with the code that called it. A `SUB` or a `FUNCTION`, with parameters and variables of its own, is nearly always the better tool.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or that label. Found when the program is compiled
3	`RETURN without GOSUB`	A `RETURN` ran with no `GOSUB` to go back to
81	`out of stack space`	Too many `GOSUB`s without a `RETURN`

NOTES

A `GOSUB` inside a `SUB` or `FUNCTION` can only reach lines of that same procedure.

SEE ALSO

`RETURN`, `GOTO`, `ON`, `SUB`, `FUNCTION`.

EXAMPLE

```
> 10 GOSUB 100 : PRINT "back" : END
> 100 PRINT "in the subroutine" : RETURN
> RUN
in the subroutine
back
```

GOTO

A statement: goes to another line.

SYNTAX

```
GOTO {line | label}
```

DESCRIPTION

Control goes to the first statement of the line with that number or that label, and carries on from there.

The same jump is written inside an `IF` as `IF c THEN 100` or `IF c GOTO 100`.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	No line has that number or that label. Found when the program is compiled

NOTES

A label marks a line that has no number. In a program typed with line numbers, the targets are numbers; in a program written in a file without them, they are labels, and a line may still be given a number so that an old listing's `GOTO 100` works.

`GOTO` typed at the prompt can't jump into the program: a line typed there is compiled alone, and the program's lines are not known to it (`label not defined`).

SEE ALSO

`GOSUB`, `ON`, `IF`, `DO`, `EXIT`.

HEX$

A string function: a number written in hexadecimal.

SYNTAX

```
HEX$(x)
```

ARGUMENTS

x

A numeric expression of any kind.

RESULT

The text of *x* in base 16, as `STR$(x, "x")` would give it, with no space in front. The digits above 9 are the capitals A to F.

This is the number itself written in another base, not the contents of a machine word. A negative number has a minus sign, so `HEX$(-1)` is `-1`, where QuickBASIC gives `FFFF`. A number that isn't an integer is written as what it is: 2.5 is `2.8` (two and eight sixteenths), and 1/3 is `1/3`. An integer can be of any length.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

NOTES

For the two's complement form of a negative number in some number of bits, mask it first: `HEX$(-1 AND &HFFFF)` is `FFFF`.

To read hexadecimal text back, put `&H` before it and use `VAL`.

SEE ALSO

`OCT$`, `STR$`, `BASE`, `VAL`.

EXAMPLE

```
> PRINT HEX$(255); " "; HEX$(-1); " "; HEX$(2 ^ 40); " "; HEX$(2.5); " "; HEX$(-1 AND
&HFFFF)
FF -1 10000000000 2.8 FFFF
```

IF

A statement: runs statements only if a condition holds. It has a one-line form and a block form.

SYNTAX

```
IF condition THEN statements [ELSE statements]
IF condition THEN line
IF condition GOTO line

IF condition THEN
    [statements]
[ELSEIF condition THEN
    [statements]] ...
[ELSE
    [statements]]
END IF
```

ARGUMENTS

condition

A numeric expression. It holds if its value is not 0. A comparison gives –1 or 0, but any number will do: 1/2 holds.

line

A line number or a label, which is jumped to.

DESCRIPTION

The one-line form is all on one line. If the condition holds, the statements after `THEN` run, up to the `ELSE` or the end of the line; if it doesn't, the statements after `ELSE` run, if there is an `ELSE`. Either part can be several statements with colons between them, and they all belong to the `IF`: nothing on the line after `THEN` runs when the condition fails, except what follows `ELSE`.

A one-line `IF` can hold another. An `ELSE` belongs to the nearest `IF` before it that has none.

The block form has nothing after `THEN` on the first line. The conditions are tested in order, the `IF`'s and then each `ELSEIF`'s, and the statements under the first one that holds are run; the rest are skipped, their conditions not evaluated. If none holds, the statements under `ELSE` run. Control then goes on after `END IF`.

Block `IF`s nest to any depth, and can hold loops and other blocks, each closed inside the part it began in.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	The condition is a string. (Two strings can be compared: the comparison is a number)
80	`block without its start or end`	A block `IF` with no `END IF`; `ELSE`, `ELSEIF` or `END IF` with no block `IF`
8	`label not defined`	`THEN` or `GOTO` with a line that isn't there

NOTES

At the prompt only the one-line form can be used, since a line typed there is compiled alone.

Both sides of an `AND` or an `OR` in a condition are always evaluated. To test one thing only if another holds, use two `IF`s.

SEE ALSO

`SELECT`, `AND`, `OR`, `NOT`, `GOTO`.

EXAMPLE

```
> 10 IF 1 THEN PRINT "a"; : PRINT "b" ELSE PRINT "c"; : PRINT "d"
> 20 IF 0 THEN PRINT "a"; : PRINT "b" ELSE PRINT "c"; : PRINT "d"
> 30 IF 5 THEN IF 0 THEN PRINT "x" ELSE PRINT "inner else"
> 40 X = 3
> 50 IF X = 1 THEN
> 60   PRINT "one"
> 70 ELSEIF X = 3 THEN
> 80   PRINT "three"
> 90 ELSE
> 100  PRINT "other"
> 110 END IF
> RUN
ab
cd
inner else
three
```

IMAG

A numeric function: the imaginary part of a number.

SYNTAX

```
IMAG(z)
```

ARGUMENTS

z

A numeric expression of any kind.

RESULT

The imaginary part of *z*, a real number of the kind it has in *z*. 0 if *z* is real.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*z* is a string

SEE ALSO

`REAL`, `COMPLEX`.

EXAMPLE

```
> PRINT IMAG(1+2i); REAL(1+2i); IMAG(5); IMAG(1/2+0.5i)
 2  1  0  0.5
```

IMP

An operator: bitwise implication.

SYNTAX

```
a IMP b
```

RESULT

An integer. Each bit of it is 0 where that bit is 1 in *a* and 0 in *b*, and 1 everywhere else. It is `(NOT a) OR b`.

For two truth values, −1 and 0, the result is false only when *a* is true and *b* is false: "if *a*, then *b*".

The operands are evaluated, rounded and treated as `AND`'s are: see `AND`. `IMP` binds less tightly than every other operator.

EXAMPLE

```
> PRINT -1 IMP 0; 0 IMP 0; 0 IMP -1; -1 IMP -1; 5 IMP 3
 0 -1 -1 -1 -5
```

INCLUDE

A statement: reads another file's lines into the program at this point.

SYNTAX

```
INCLUDE "file"
'$INCLUDE: 'file'
REM $INCLUDE: 'file'
```

ARGUMENTS

file

The file's name, written out: a literal, not an expression.

DESCRIPTION

The lines of *file* are compiled as if they were written where the `INCLUDE` line is. The statement must be on a line of its own. The second and third forms are QuickBASIC's, in which the statement is disguised as a comment; they do the same.

A name that starts with `/` is used as it is. Any other name is looked for in the directory the program itself came from, and then in `/lib/basic`.

An included file can include others, four deep. A program can include eight files in all.

An error in an included file is reported with that file's name and its own line number there.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	The file is in neither place
12	`illegal in direct mode`	`INCLUDE` typed at the prompt
2	`syntax error`	`INCLUDE` with other statements on its line

NOTES

Including is done when the program is compiled, not when control reaches the line, so a file can't be chosen by the running program. To run another program from a program, use `SHELL "basic file"`.

SEE ALSO

`LOAD`, `SHELL`, `DECLARE`.

EXAMPLE

```
% echo 'PRINT "included"' >inc.bas
% echo 'PRINT "main"' >main.bas
% echo 'INCLUDE "inc.bas"' >>main.bas
% echo 'PRINT "after"' >>main.bas
% basic main.bas
main
included
after
```

INKEY$

A string function: a key that has been pressed, without waiting for one.

SYNTAX

```
INKEY$
```

RESULT

The empty string if no key is waiting. Otherwise one key, which is taken out of the keyboard's queue:

- An ordinary key is a string of one byte, the key's character.
- A cursor key or a function key is a string of two bytes: `CHR$(0)`, then a code for the key. The codes are QuickBASIC's: 72 is up, 80 is down, 59 is F1, and so on.

DESCRIPTION

`INKEY$` never waits. The usual use is in a loop that does something else as well, or one that waits for any key: `DO : K$ = INKEY$ : LOOP UNTIL K$ <> ""`.

NOTES

A loop that only polls `INKEY$` takes all the processor time it can get. Put a `SLEEP` of a short time in it if other tasks are running.

To wait for a set number of keys, use `INPUT$`.

SEE ALSO

`INPUT$`, `INPUT`, `LINE INPUT`, `SLEEP`.

INPUT

A statement: reads a line and assigns its fields to variables. The line comes from the console, or from a file.

SYNTAX

```
INPUT [;] ["prompt" {; | ,}] v [, v] ...
INPUT #f, v [, v] ...
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. It must be a literal: a string variable there is a syntax error. After it, `;` adds a question mark and a space, and `,` adds nothing. With no prompt at all, `?` is written.

v

A numeric or string variable, or an array's element. Each takes one field of the line.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

From the console. The prompt is written and one line is read, with the console's line editing (section 13.2). Only its first 255 bytes are used. The line is split into fields at its commas, and each variable takes a field, in order:

- A numeric variable's field is read as a number, in any form a program's own numbers may take: `12`, `-7/2`, `2.5`, `1e3`, `&HFF`. An empty field is 0.
- A string variable's field is the text up to the next comma, with the spaces before and after it dropped. A field that starts with a double quote runs to the closing quote instead, and can hold commas and spaces.

If the line has fewer fields than there are variables, or more, or a numeric field that isn't a number, HyBASIC writes `?Redo from start` and the statement starts again from its prompt. It is never an error.

From a file. One line of file *f* is read, and split and assigned in the same way, with three differences. There is no prompt and nothing is asked again: a numeric field that isn't a number is 0. If the line runs out of fields before the variables do, the next line is read and its fields carry on. Fields left on the line after the last variable is filled are dropped: the next `INPUT #` starts at the next line.

The semicolon after `INPUT` matters only when input doesn't come from the console (`basic prog <answers`). The console echoes the Return that ends a line; a file doesn't, so HyBASIC writes a new line itself after reading one. `INPUT ;` leaves it out, and what is printed next follows the prompt on the same line.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`
2	`syntax error`	The prompt isn't a string literal

CTRL-C at the prompt stops the program, as it does anywhere else (`Break`).

NOTES

To read a whole line, commas and all, into one string, use `LINE INPUT`. To read keys without waiting for Return, use `INKEY$` or `INPUT$`.

SEE ALSO

`LINE INPUT`, `INPUT$`, `INKEY$`, `OPEN`, `EOF`, `READ`.

EXAMPLE

```
> 10 INPUT "Two numbers"; A, B
> 20 PRINT A + B
> 30 INPUT "Name, age: ", N$, G
> 40 PRINT "["; N$; "]"; G
> 50 INPUT X, Y$
> 60 PRINT X; "["; Y$; "]"
> RUN
Two numbers? 3
?Redo from start
Two numbers? 1/2, 2.5
 3
Name, age: Sam Smith , 12
[Sam Smith] 12
? , "x, y"
 0 [x, y]
```

Fields that carry on from one line of a file to the next:

```
> 10 OPEN "/ram/n.txt" FOR OUTPUT AS #1
> 20 PRINT #1, "1" : PRINT #1, "2, 3" : PRINT #1, "4"
> 30 CLOSE #1 : OPEN "/ram/n.txt" FOR INPUT AS #1
> 40 INPUT #1, A, B : PRINT A; B
> 50 INPUT #1, C : PRINT C
> 60 INPUT #1, D
> RUN
 1  2
 4
line 60: input past end of file
```

INPUT$

A string function: a set number of keys from the keyboard, or of bytes from a file.

SYNTAX

```
INPUT$(n)
INPUT$(n, [#]f)
```

ARGUMENTS

n

A numeric expression: how many keys or bytes.

f

A numeric expression: the number of a file opened `FOR INPUT` or `FOR BINARY`.

RESULT

From the keyboard, `INPUT$(n)` waits until *n* keys have been typed and returns them as a string of *n* bytes. Nothing is echoed and Return is a key like any other, so this is the way to read a key press, a password, or a menu choice without the user pressing Return. When standard input is not a console, it

reads *n* bytes from it instead.

From a file, `INPUT$(n, #f)` returns the next *n* bytes of file *f*, whatever they are, line ends included, and moves the file's position past them.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	The file, or standard input, ends before *n* bytes
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`

NOTES

To read a whole file into a string, ask for its length: `INPUT$(LOF(1), #1)`.

SEE ALSO

`INKEY$`, `INPUT`, `LINE INPUT`, `GET`, `LOF`.

EXAMPLE

```
> OPEN "/ram/t.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : CLOSE
> OPEN "/ram/t.txt" FOR INPUT AS #1 : PRINT INPUT$(3, #1); "|"; INPUT$(2, 1); "|";
LOC(1)
abc|de| 5
> CLOSE
```

INSTR

A numeric function: where one string is found inside another.

SYNTAX

```
INSTR([start,] s$, t$)
```

ARGUMENTS

start

A numeric expression, an integer from 1 up: the position in *s$* to start looking from. 1 if it is left out.

s$

A string expression: the string to look in.

t$

A string expression: the string to look for.

RESULT

The position in *s$*, counting its first byte as 1, of the first place at or after *start* where *t$* appears whole. 0 if it appears nowhere from there on, which includes the cases where *start* is beyond the end of *s$* and where *t$* is longer than what is left.

If *t$* is empty, the result is *start*, as long as *start* is within *s$*.

The comparison is of bytes: capitals and small letters are different.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*start* is less than 1, or is not an integer
13	`type mismatch`	*s$* or *t$* is a number

NOTES

To find something without regard to case, compare capitals: `INSTR(UCASE$(s$), UCASE$(t$))`.

To find every place, call it again with *start* one past the last result.

SEE ALSO

`MID$`, `LEFT$`, `RIGHT$`, `LEN`.

EXAMPLE

```
> PRINT INSTR("hello", "l"); INSTR(4, "hello", "l"); INSTR("hello", "z"); INSTR("Hello",
"h")
 3  4  0  0
> PRINT INSTR("hello", ""); INSTR(9, "hello", "l")
 1  0
```

INT

A numeric function: a real number rounded down.

SYNTAX

```
INT(x)
```

ARGUMENTS

x

A numeric expression whose value is real.

RESULT

The largest integer that is not more than *x*. For a negative number that isn't whole this is the next integer *below* it: `INT(-2.5)` is –3. The result is an exact integer of whatever length.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex
13	`type mismatch`	*x* is a string

NOTES

The three ways to make an integer differ only for numbers that aren't whole: `INT` rounds down, `FIX` rounds toward zero, and `CINT` rounds to the nearest.

SEE ALSO

`FIX`, `CINT`, `MOD`, `FIXED`.

EXAMPLE

```
> PRINT INT(2.5); INT(-2.5); INT(7/2); INT(-7/2); INT(9)
 2 -3  3 -4  9
```

IS

Part of `SELECT CASE`: `CASE IS < 10` tests the selected value with a comparison. See `SELECT`.

KILL

A statement: removes a file.

SYNTAX

```
KILL path$
```

ARGUMENTS

path$

A string expression: the file's name. It is one name, not a pattern.

DESCRIPTION

The file is removed from its directory, and the space it took is free. There is no way to get it back.

A directory can be removed with `KILL` too, if it is empty, though `RMDIR` says more clearly what is meant.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no such file
263	`busy`	The file is open, in this program or any other
70	`permission denied`	The file is read-only, or its disk is
13	`type mismatch`	*path$* is a number

SEE ALSO

`NAME`, `RMDIR`, `FILES`, `OPEN`.

EXAMPLE

```
> OPEN "/ram/u.txt" FOR OUTPUT AS #1 : KILL "/ram/u.txt"
busy
> CLOSE : KILL "/ram/u.txt" : KILL "/ram/u.txt"
file not found
```

LBOUND, UBOUND

Numeric functions: the lowest and the highest index of an array.

SYNTAX

```
LBOUND(array [, dimension])
UBOUND(array [, dimension])
```

ARGUMENTS

array

An array's name, with no parentheses.

dimension

A numeric expression: which dimension, counting the first as 1. 1 if it is left out.

RESULT

The lowest, or the highest, index the array allows in that dimension, as `DIM` set them.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*dimension* is less than 1 or more than the array has

NOTES

If the name has not been used as an array, asking for its bounds makes it one, with the usual indexes 0 to 10: `UBOUND` of a misspelled name is 10, not an error.

A loop over a whole array is best written `FOR I = LBOUND(A) TO UBOUND(A)`, which goes on being right if the `DIM` is changed.

SEE ALSO

`DIM`, `REDIM`, `OPTION`.

EXAMPLE

```
> DIM A(2 TO 5, 7) : PRINT LBOUND(A); UBOUND(A); LBOUND(A, 2); UBOUND(A, 2)
 2  5  0  7
```

LCASE$, UCASE$

String functions: a string with its letters made small, or made capital.

SYNTAX

```
LCASE$(s$)
UCASE$(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

A string of the same length. `LCASE$` turns each of the capitals A to Z into its small letter; `UCASE$` turns each of a to z into its capital. Every other byte is copied as it is, including the accented letters above 127 in the console's character sets.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

SEE ALSO

`INSTR`, `LTRIM$`.

EXAMPLE

```
> PRINT LCASE$("Hello World 123"); " "; UCASE$("Hello World 123")
hello world 123 HELLO WORLD 123
```

LEFT$, RIGHT$

String functions: the first, or the last, bytes of a string.

SYNTAX

```
LEFT$(s$, n)
RIGHT$(s$, n)
```

ARGUMENTS

s$

A string expression.

n

A numeric expression, 0 or more: how many bytes. One that isn't an integer is rounded to the nearest, a half going to the even one.

RESULT

`LEFT$` is the first *n* bytes of *s$*, and `RIGHT$` the last *n*. If *s$* has fewer than *n* bytes, the result is all of it. If *n* is 0, the result is the empty string.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*n* is negative
13	`type mismatch`	*s$* is a number, or *n* a string

SEE ALSO

`MID$`, `LEN`, `INSTR`.

EXAMPLE

```
> PRINT "["; LEFT$("hello", 2); "]["; RIGHT$("hello", 2); "]["; LEFT$("hello", 9); "][";
RIGHT$("hi", 0); "]"
[he][lo][hello][]
```

LEN

A numeric function: the length of a string.

SYNTAX

```
LEN(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

The number of bytes in *s$*: 0 for the empty string. Every byte counts, zero bytes and line ends included.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number. `LEN` does not give the size of a numeric variable

NOTES

`LEN = n` at the end of an `OPEN` statement is a different thing: see `OPEN`.

SEE ALSO

`LEFT$`, `MID$`, `INSTR`, `STR$`.

EXAMPLE

```
> PRINT LEN("hello"); LEN(""); LEN(CHR$(0) + CHR$(0)); LEN(STRING$(1000, "x"))
 5  0  2  1000
```

LET

A statement: gives a variable a value.

SYNTAX

```
[LET] variable = expression
```

ARGUMENTS

variable

A variable, an array's element, a record or one of its fields.

expression

An expression of the variable's type.

DESCRIPTION

The expression is evaluated, and its value becomes the variable's. The word `LET` is optional and changes nothing.

A numeric variable takes any number as it is: an integer of any length, a rational, a decimal or a complex number. Nothing is rounded or converted on the way in. A string variable takes a string of any length up to 64K.

Assigning to a function's own name, inside the function, sets the value it will return.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	A string for a numeric variable, or a number for a string variable
10	`duplicate definition`	The name is a constant's
9	`subscript out of range`	An array's element that isn't there

NOTES

= is also the comparison "is equal to". Which one is meant depends on where it stands: the first = of an assignment assigns, and any after it compare, so `A = B = C` gives `A` the truth of `B = C`.

A line typed at the prompt that starts with a number is taken as a line of the program, whatever follows the number.

SEE ALSO

`CONST`, `DIM`, `SWAP`, `READ`.

EXAMPLE

```
> B = 2 : C = 2 : A = B = C : PRINT A : A = B = 3 : PRINT A
-1
 0
> LET N = 2 ^ 100 / 3 : PRINT N
 12676506002282294014967032053 76/3
```

LINE INPUT

A statement: reads a whole line into a string variable.

SYNTAX

```
LINE INPUT [;] ["prompt";] v$
LINE INPUT #f, v$
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. Nothing is added to it: no question mark.

v$

A string variable, or an array's element.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

One line is read and assigned to *v$* as it stands: commas, quotes and spaces at either end are all kept, where `INPUT` would split the line at its commas and trim it. The line's end is not part of the string. A line longer than 255 bytes is cut to 255.

From the console the line is typed with the console's line editing (section 13.2). From a file, the next line of the file is read, and the file's position moves past its line end.

The semicolon after `INPUT` has the meaning it has for `INPUT`: it matters only when input is not a console.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
13	`type mismatch`	The variable is numeric
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`

NOTES

`LINE` with an opening parenthesis after it is the graphics statement that draws a line.

SEE ALSO

INPUT, INPUT$, EOF, OPEN.

EXAMPLE

```
> Q$ = CHR$(34) : OPEN "/ram/t.txt" FOR OUTPUT AS #1 : PRINT #1, "  two, "; Q$; "words";
Q$; "  " : CLOSE
> OPEN "/ram/t.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT "["; L$; "]"; EOF(1)
[  two, "words"  ]-1
> LINE INPUT #1, L$
input past end of file
```

LIST

A command of the prompt: shows the program's lines.

SYNTAX

```
LIST [from][-[to]]
```

ARGUMENTS

from, to

Line numbers or labels.

DESCRIPTION

TYPED	LISTS
LIST	The whole program
LIST 100	Line 100 alone
LIST 100-	From line 100 to the end
LIST -100	From the start to line 100
LIST 100-200	Lines 100 to 200

A number need not be a line that exists: the lines in the range are listed, and if there are none nothing is. The lines are shown as they were typed or loaded, with their spacing and their case.

CTRL-C stops a long listing.

ERRORS

ERR	MESSAGE	CAUSE
89	only at the prompt	LIST in a program's line

SEE ALSO

EDIT, DELETE, SAVE, LOAD.

EXAMPLE

```
> 10 PRINT "one"
> 20 PRINT "two"
> 30 PRINT "three"
> LIST 20-
20 PRINT "two"
30 PRINT "three"
> LIST 15-25
20 PRINT "two"
```

LOAD

A command of the prompt: reads a program from a file, in place of the one in memory.

SYNTAX

```
LOAD "file"
```

ARGUMENTS

file

The file's name, in quotes. Nothing is added to it: a program saved as `game.bas` is loaded as `"game.bas"`.

DESCRIPTION

The program in memory is dropped first, with its variables and any files it had open, as `NEW` does; then the file's lines become the program. The file is plain text, a line of the program to each line, with or without line numbers. A carriage return before a line feed is dropped, so a file written on a PC loads as it is.

The program is not run. `RUN` runs it; `basic file`, typed at the shell, loads and runs a file in one step.

ERRORS

ERR	MESSAGE	CAUSE
288	`not found`	There is no such file. The message is preceded by the file's name
89	`only at the prompt`	`LOAD` in a program's line

NOTES

The old program is dropped before the file is opened. If the `LOAD` then fails, there is no program in memory at all: `SAVE` first.

SEE ALSO

`SAVE`, `NEW`, `RUN`, `INCLUDE`, `EDIT`.

EXAMPLE

```
> 10 PRINT "saved"
> SAVE "/ram/s.bas"
> NEW
> LOAD "/ram/s.bas"
> LIST
10 PRINT "saved"
> LOAD "/ram/nothere.bas"
/ram/nothere.bas: not found
> LIST
```

LOC, LOF

Numeric functions: an open file's position, and its length.

SYNTAX

```
LOC(f)
LOF(f)
```

ARGUMENTS

f

A numeric expression: the number of an open file, in any mode.

RESULT

`LOC(f)` is the number of bytes before the file's position: how many have been read or written so far, in a file used from start to end. It is 0 when a file has just been opened, except `FOR APPEND`, where it is the file's length.

`LOF(f)` is the file's length in bytes, as it is now, including what this program has written to it.

`SEEK(f)` is `LOC(f) + 1`: the position counted from 1, which is the form the `SEEK` and `GET` statements take.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open

SEE ALSO

`SEEK`, `EOF`, `OPEN`, `GET`, `PUT`.

EXAMPLE

```
> OPEN "/ram/l.txt" FOR OUTPUT AS #1 : PRINT #1, "abcdef" : PRINT LOC(1); LOF(1);
SEEK(1) : CLOSE
 7  7  8
```

LOCATE

A statement: moves the cursor, and shows or hides it.

SYNTAX

```
LOCATE [row] [, [column] [, cursor]]
```

ARGUMENTS

row, column

Numeric expressions, from 1: the top line of the screen is row 1 and its left edge column 1. Each is rounded to the nearest integer. One that is left out keeps its present value.

cursor

A numeric expression: 0 hides the cursor and 1 shows it.

DESCRIPTION

The cursor is moved by writing an escape sequence to standard output (section 13.7), and the next thing printed starts there. `POS(0)` follows the move.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*row* or *column* is less than 1, or more than 255
13	`type mismatch`	An argument is a string

NOTES

The screen's real size isn't checked. A position beyond the edge of the terminal is the terminal's to deal with; most stop at the edge.

SEE ALSO

`CSRLIN`, `POS`, `CLS`, `PRINT`, `TAB`.

EXAMPLE

The transcript shows what was printed, not where on the screen it went. `POS(0)` is read after `CSRLIN` has been printed, so it is 17, not 14:

```
> LOCATE 5, 10 : PRINT "here"; : PRINT CSRLIN; POS(0)
here 5  17
```

LOG

A numeric function: the natural logarithm.

SYNTAX

```
LOG(x)
```

ARGUMENTS

x

A numeric expression whose value is real and not 0.

RESULT

The logarithm of *x* to the base e. `LOG(1)` is the integer 0. Any other result is a decimal, rounded to the number of significant digits that `DIGITS` has set (12 unless it has been changed).

For a negative *x* the result is complex: the logarithm of the size of *x*, plus π i. `LOG(-1)` is π i.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is 0, or is complex
13	`type mismatch`	*x* is a string

NOTES

A logarithm to another base is a quotient: `LOG(x) / LOG(10)` is the logarithm to base 10.

SEE ALSO

`EXP`, `SQR`, `DIGITS`.

EXAMPLE

```
> PRINT LOG(1); LOG(10); LOG(1/2); LOG(2 ^ 100); LOG(-1)
 0  2.30258509299 -0.69314718056  69.314718056  3.14159265359i
> PRINT LOG(0)
illegal function call
```

LOOP

The line that ends a `DO` loop, with or without a condition. See `DO`.

LTRIM$, RTRIM$

String functions: a string without the spaces at its start, or at its end.

SYNTAX

```
LTRIM$(s$)
RTRIM$(s$)
```

ARGUMENTS

s$

A string expression.

RESULT

`LTRIM$` is *s$* from its first byte that is not a space; `RTRIM$` is *s$* up to its last byte that is not a space. Only the space, byte 32, is removed: tabs and line ends are kept. A string that is all spaces gives the empty string.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*s$* is a number

NOTES

`STR$` puts a space in front of a number that isn't negative. `LTRIM$(STR$(n))` is the number's digits alone.

SEE ALSO

`STR$`, `LEFT$`, `RIGHT$`, `SPACE$`.

EXAMPLE

```
> PRINT "["; LTRIM$("  a b  "); "]["; RTRIM$("  a b  "); "]["; LTRIM$("   "); "]";
LTRIM$(STR$(42)); "|"
[a b  ][  a b][]42|
```

MID$

A string function, and a statement: a part of a string, taken out or replaced.

SYNTAX

```
MID$(s$, start [, length])
MID$(v$, start [, length]) = new$
```

ARGUMENTS

s$

A string expression.

v$

A string variable, or an array's element.

start

A numeric expression, 1 or more: the position of the first byte, counting the string's first byte as 1.

length

A numeric expression, 0 or more: how many bytes. If it is left out, all the rest of the string.

new$

A string expression.

Numbers that aren't integers are rounded to the nearest, a half going to the even one.

RESULT

The function is the *length* bytes of *s$* that start at *start*, or as many as there are if the string ends sooner. It is the empty string if *start* is past the end, or *length* is 0.

DESCRIPTION

The statement overwrites part of the variable's string with the bytes of *new$*, starting at *start*. It never changes the string's length. The number of bytes replaced is the smallest of three: *length* (if it is given), the length of *new$*, and what is left of *v$* from *start* to its end. Bytes of *new$* that don't fit are not used.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*start* is less than 1, or *length* is negative; in the statement, *start* is past the end of *v$*
13	`type mismatch`	A number where a string is wanted, or a string where a number is

SEE ALSO

`LEFT$`, `RIGHT$`, `INSTR`, `LEN`.

EXAMPLE

```
> PRINT "["; MID$("hello", 2, 3); "]["; MID$("hello", 4); "]["; MID$("hello", 4, 9); "]
["; MID$("hello", 9); "]"
[ell][lo][lo][]
> A$ = "hello" : MID$(A$, 2, 2) = "XYZ" : PRINT A$ : MID$(A$, 4) = "12345" : PRINT A$;
LEN(A$)
hXYlo
hXY12 5
```

MKDIR, RMDIR

Statements: make a directory, and remove one.

SYNTAX

```
MKDIR path$
RMDIR path$
```

ARGUMENTS

path$

A string expression: the directory's name.

DESCRIPTION

`MKDIR` makes a new, empty directory. The directory it is to go in must exist already: one level is made at a time.

`RMDIR` removes a directory, which must be empty.

ERRORS

ERR	MESSAGE	CAUSE
58	`file already exists`	`MKDIR`: there is a file or a directory of that name
53	`file not found`	`MKDIR`: the directory it would go in isn't there. `RMDIR`: there is no such directory
292	`directory not empty`	`RMDIR`: the directory still has something in it
70	`permission denied`	The disk can't be written

SEE ALSO

`CHDIR`, `FILES`, `KILL`, `NAME`.

EXAMPLE

```
> MKDIR "/ram/d1" : MKDIR "/ram/d1/sub" : RMDIR "/ram/d1"
directory not empty
> RMDIR "/ram/d1/sub" : RMDIR "/ram/d1" : RMDIR "/ram/d1"
file not found
```

MKN$

A string function: a number's bytes in the stored format.

SYNTAX

```
MKN$(x)
```

ARGUMENTS

x

A numeric expression of any kind.

RESULT

A string holding *x* in the form numbers are stored in: the form `PUT` writes for a numeric variable, and `CVN` reads back. Its length depends on the number. A small integer takes one byte; larger integers, rationals, decimals and complex numbers take as many as they need.

DESCRIPTION

`MKN$` and `CVN` are the pair that stands in place of QuickBASIC's `MKI$`, `MKL$`, `MKS$`, `MKD$` and their `CV` functions. `CVN(MKN$(x))` is exactly *x*, for every number.

Because the length varies, a number can't be given a fixed place in a record by `MKN$` alone. Pad the string to the width wanted, or keep the number as text.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	*x* is a string

SEE ALSO

CVN, PUT, GET, STR$.

EXAMPLE

```
> PRINT LEN(MKN$(7)); LEN(MKN$(1/3)); LEN(MKN$(2 ^ 100)); CVN(MKN$(2 ^ 100))
 1  3  14  126765060022822940149670320 5376
```

MOD

An operator: the remainder of an integer division. The entry covers `\`, integer division, too.

SYNTAX

```
a MOD b
a \ b
```

ARGUMENTS

a, b

Numeric expressions whose values are real.

RESULT

Both operands are first rounded to the nearest integer, a half going to the even one. Then `a \ b` is the quotient with its fraction dropped, which rounds toward zero, and `a MOD b` is what is left over: `a - (a \ b)

- b`. The remainder has the sign of *a*, or is 0.

The integers can be of any length.

DESCRIPTION

`\` binds more tightly than `MOD`, and both less tightly than `*` and `/` and more tightly than `+` and –. So `A + B MOD C \ D` is `A + (B MOD (C \ D))`.

ERRORS

ERR	MESSAGE	CAUSE
11	`division by zero`	*b* is 0, or rounds to 0
13	`type mismatch`	An operand is a string

NOTES

Because the operands are rounded first, `7.5 MOD 2` is 0: 7.5 rounds to 8. For the remainder of a division of numbers that aren't whole, work it out: `x - INT(x / y) * y`.

`/` is exact division, and gives a rational when the integers don't divide: `7 / 2` is 7/2.

SEE ALSO

INT, FIX, GCD.

EXAMPLE

```
> PRINT 7 MOD 3; -7 MOD 3; 7 MOD -3; 7 \ 2; -7 \ 2; 7.5 MOD 2; 7 / 2
 1 -1  1  3 -3  0  7/2
> PRINT 2 ^ 70 MOD 1000; 2 + 3 * 4 ^ 2 MOD 5 \ 2
 424  2
```

NAME

A statement: renames a file.

SYNTAX

```
NAME old$ AS new$
```

ARGUMENTS

old$

A string expression: the file's present name.

new$

A string expression: its new name.

DESCRIPTION

The file, or directory, *old$* is given the name *new$* in the directory it is already in. Only the last part of *new$* is used, the name after its last `/`: a rename can't move a file to another directory, and a directory written in *new$* is ignored without an error. To move a file, copy it and remove the original.

Renaming a file to the name it has does nothing.

ERRORS

ERR	MESSAGE	CAUSE
53	`file not found`	There is no *old$*
58	`file already exists`	There is a *new$* in that directory already
70	`permission denied`	The disk can't be written

SEE ALSO

KILL, FILES, MKDIR, SHELL.

EXAMPLE

```
> OPEN "/ram/n1" FOR OUTPUT AS #1 : CLOSE : MKDIR "/ram/dd"
> NAME "/ram/n1" AS "/ram/dd/moved" : FILES "/ram" : FILES "/ram/dd"
bin/            lib/            moved           dd/
```

NEW

A command of the prompt: drops the program.

SYNTAX

```
NEW
```

DESCRIPTION

The program's lines are removed, with everything that belonged to it: its variables and arrays, its procedures and labels, and any files it had open, which are closed. A stopped program is gone, so there is nothing to `CONT`. The settings that are HyBASIC's and not the program's stay as they are: `BASE`, `DIGITS`, the current directory.

There is no question first, and no way back. `SAVE` what is wanted.

ERRORS

ERR	MESSAGE	CAUSE
89	`only at the prompt`	`NEW` in a program's line

SEE ALSO

`LOAD`, `SAVE`, `CLEAR`, `DELETE`.

NEXT

The line that ends a `FOR` loop. See `FOR`.

NOT

An operator: the bitwise complement of an integer, which is also the logical NOT of a truth value.

SYNTAX

```
NOT a
```

RESULT

An integer: a, rounded to the nearest integer if it isn't one, with every bit turned over. In two's complement that is $-a-1$: `NOT 0` is −1, `NOT -1` is 0, and `NOT 5` is −6.

For the truth values −1 and 0, `NOT` gives the other one. For any other number it does not: 5 counts as true in an `IF`, and so does `NOT 5`.

DESCRIPTION

`NOT` binds less tightly than the comparisons and more tightly than `AND`: `NOT A = B` is `NOT (A = B)`, and `NOT A AND B` is `(NOT A) AND B`.

The operand is rounded and treated as `AND`'s are: see `AND`.

EXAMPLE

```
> PRINT NOT 0; NOT -1; NOT 5; NOT 1 = 2
-1  0 -6 -1
```

NUMERATOR

A numeric function: the numerator of a real number written as a fraction in lowest terms.

SYNTAX

```
NUMERATOR(x)
```

RESULT

An integer with the sign of *x*. A rational gives its numerator, an integer gives itself, and a decimal is turned into the fraction it exactly equals first: −0.75 is −3/4, and gives −3. See `DENOMINATOR`, which has the errors and an example.

OCT$

A string function: a number written in octal.

SYNTAX

```
OCT$(x)
```

RESULT

The text of *x* in base 8, with no space in front, as `STR$(x, "o")` would give it. As with `HEX$`, it is the number itself in another base: a negative number has a minus sign, and a number that isn't an integer is written as what it is. See `HEX$`.

EXAMPLE

```
> PRINT OCT$(8); " "; OCT$(255); " "; OCT$(-8); " "; OCT$(0.5)
10 377 -10 0.4
```

ON

A statement: jumps to one of several lines by a number; or sets where errors go.

SYNTAX

```
ON n GOTO line [, line] ...
ON n GOSUB line [, line] ...
ON ERROR GOTO line
ON ERROR GOTO 0
```

ARGUMENTS

n

A numeric expression, rounded to the nearest integer.

line

A line number or a label.

DESCRIPTION

ON n GOTO goes to the first line in the list if *n* is 1, to the second if it is 2, and so on. If *n* is less than 1 or more than the number of lines listed, nothing happens: the program goes on with the next statement. **ON n GOSUB** is the same, but calls the line as a subroutine, and its RETURN comes back to the statement after the ON.

ON ERROR GOTO line names an error handler. From then on, an error that would have stopped the program makes it go to *line* instead, with ERR, ERL and ERR$ set. The handler ends with a RESUME, which says where to carry on: at the statement that failed, at the one after it, or at another line.

While the handler is running, a second error is not sent to it again. It stops the program in the ordinary way.

ON ERROR GOTO 0 turns the handler off. Used inside a handler, it gives up: the error being handled stops the program, with its message, as if there had been no handler.

ERRORS

ERR	MESSAGE	CAUSE
8	`label not defined`	A line in the list, or the handler's, isn't in the program
13	`type mismatch`	*n* is a string

NOTES

CTRL-C is not an error, and no handler sees it.

SEE ALSO

RESUME, ERR, ERROR, GOTO, GOSUB, SELECT.

EXAMPLE

```
> 10 ON ERROR GOTO 100
> 20 X = 1 / 0 : PRINT "after the error"
> 30 OPEN "/ram/nothere" FOR INPUT AS #1
> 40 PRINT "line 40" : ON ERROR GOTO 0 : ERROR 5
> 50 PRINT "not reached"
> 100 PRINT "handler:"; ERR; ERL
> 110 IF ERR = 11 THEN RESUME NEXT
> 120 RESUME 40
> RUN
handler: 11  20
after the error
handler: 53  30
line 40
line 40: illegal function call
```

OPEN

A statement: opens a file, and gives it a number for the statements that read and write it.

SYNTAX

```
OPEN path$ FOR {INPUT | OUTPUT | APPEND | BINARY} AS [#]f [LEN = n]
```

ARGUMENTS

path$

A string expression: the file's name. A name that doesn't start with `/` is taken from the current directory (`CHDIR`). It can be any name in the program's namespace: a file on a disk, or a device such as `/dev/cons`.

f

A numeric expression from 1 to 255: the number the file is known by until it is closed. The `#` is optional.

n

Accepted, evaluated and ignored. QuickBASIC's record length has no meaning here: `GET` and `PUT` take a position in bytes.

DESCRIPTION

The mode decides how the file is opened, and which statements may then use it:

MODE	A FILE THAT EXISTS	A FILE THAT DOESN'T	STARTS AT	MAY THEN BE USED BY
`INPUT`	Opened for reading	Error 53	Its first byte	`INPUT #`, `LINE INPUT #`, `INPUT$`, `EOF`, `LOF`, `LOC`, `SEEK`
`OUTPUT`	Emptied, and opened for writing	Made	Its first byte	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`APPEND`	Opened for writing, its contents kept	Made	Its end	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`BINARY`	Opened for reading and writing	Made	Its first byte	All of the above, and `GET` and `PUT`

Up to 8 files can be open at once. A file stays open until `CLOSE` names it, `CLOSE` with no number closes them all, or a `RUN` starts (which closes every file first). Leaving HyBASIC closes them too.

`EOF(f)` is always true (–1) for a file opened `FOR OUTPUT` or `FOR APPEND`.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	*f* is less than 1 or more than 255
55	`file already open`	Number *f* is in use
67	`too many files`	8 files are open already
53	`file not found`	`FOR INPUT`: there's no such file. Any mode: a directory in the path isn't there
70	`permission denied`	The file, or its disk (`/rom`), can't be written
13	`type mismatch`	*path$* is a number, or *f* a string

The system's other refusals come through as its own errors (a full disk is 61, `disk full`).

NOTES

The limit of 8 is HyBASIC's own. Each open file also takes one of the task's 16 file descriptors, three of which are the console's.

One file can be open under two numbers at once, each with its own position: opening it `FOR INPUT` as `#2` while it is open `FOR OUTPUT` as `#1` is allowed, and reads what has been written so far.

A file written with `PRINT #` holds exactly the bytes `PRINT` would have sent to the screen, each line ended by a line feed (byte 10).

SEE ALSO

`CLOSE`, `PRINT`, `WRITE`, `INPUT`, `LINE INPUT`, `GET`, `PUT`, `SEEK`, `EOF`, `LOF`, `LOC`, `KILL`, `NAME`.

EXAMPLE

```
> OPEN "/ram/log.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT LOF(1); EOF(1) : CLOSE
#1
 4 -1
> OPEN "/ram/log.txt" FOR APPEND AS #1 : PRINT #1, "two" : PRINT LOF(1) : CLOSE #1
 8
> OPEN "/ram/log.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT L$; EOF(1)
one 0
> PRINT #1, "three"
bad file mode
> OPEN "/ram/other.txt" FOR OUTPUT AS #1
file already open
> CLOSE : OPEN "/ram/nothere.txt" FOR INPUT AS #1
file not found
> PRINT ERR
 53
```

OPTION

A statement: sets the lowest index of arrays.

SYNTAX

```
OPTION BASE {0 | 1}
```

DESCRIPTION

An array made with only its highest index given, as in `DIM A(10)`, has indexes from 0. After `OPTION BASE 1` they are from 1, so that `DIM A(10)` has ten elements and no `A(0)`. `OPTION BASE 0` puts it back.

The setting applies to arrays made after the statement, whether by `DIM`, by `REDIM`, or by being used without either (whose indexes then run from 1 to 10). Arrays that exist already keep the bounds they were made with. A `DIM` that states its own lowest index, `DIM A(5 TO 9)`, is not affected.

The digit must be written out as 0 or 1: it is part of the statement, not an expression.

ERRORS

ERR	MESSAGE	CAUSE
2	`syntax error`	Anything but 0 or 1 after `BASE`

NOTES

This `BASE` has nothing to do with the `BASE` statement, which sets the base numbers are written in.

SEE ALSO

`DIM`, `REDIM`, `LBOUND`, `UBOUND`.

EXAMPLE

```
> OPTION BASE 1 : DIM A(3) : B(1) = 1 : PRINT LBOUND(A); UBOUND(A); LBOUND(B); UBOUND(B)
 1  3  1  10
> OPTION BASE 0 : DIM C(3) : PRINT LBOUND(C); LBOUND(A)
 0  1
```

OR

An operator: the bitwise OR of two integers, which is also the logical OR of two truth values.

SYNTAX

```
a OR b
```

RESULT

An integer. Each bit of it is 1 where the same bit is 1 in *a*, in *b*, or in both.

For two truth values, −1 and 0, the result is true if either is.

Both operands are always evaluated, and they are rounded and treated as `AND`'s are: see `AND`. `OR` binds less tightly than `AND` and more tightly than `XOR`.

EXAMPLE

```
> PRINT 5 OR 3; 0 OR 0; (1 = 2) OR (2 = 2); 1.5 OR 0
 7  0 -1  2
```

OUTPUT

A mode of `OPEN`: `OPEN path$ FOR OUTPUT AS #f` makes a file, or empties one, and opens it for writing. See `OPEN`.

PEEK, POKE

A function and a statement: read, and write, one byte of memory.

SYNTAX

```
PEEK(address)
POKE address, value
```

ARGUMENTS

address

A numeric expression: an address in HyBASIC's task, 0 to 65,535. A negative number from −32,768 to −1 is accepted too, and means the address 65,536 more: −1 is `$FFFF`.

value

A numeric expression from 0 to 255, rounded to the nearest integer.

RESULT

`PEEK` is the byte at the address, 0 to 255.

DESCRIPTION

The addresses are those of HyBASIC's own task: its zero page and stack, the RAM that holds its state, the bank window at `$8000` to `$9FFF`, the paged ROM at `$A000` to `$DFFF` and the BIOS above (Appendix F). Another task's memory can't be reached.

For an address in the bank window, the bank is the one `BANK` has set. HyBASIC puts it there for the read or the write and takes it away again, so the window is of no use to a program between the two: see `BANK`.

`POKE` does no checking beyond the two ranges. Writing into HyBASIC's own memory, which is most of what is below `$8000`, will usually stop it working; and a write to an address that is ROM does nothing. The hardware's registers are reached through the drivers' files (Chapter 12), not through `POKE`.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	The address is outside −32,768 to 65,535; or the value is outside 0 to 255
13	`type mismatch`	An argument is a string

SEE ALSO

`BANK`, `SYS`, `CALL`, `ASM`.

EXAMPLE

```
> BANK 40 : POKE &H8000, 65 : POKE &H8001, 66.5 : PRINT PEEK(&H8000); PEEK(&H8001);
PEEK(-32768)
 65  66  65
> POKE &H8000, 256
illegal function call
```

PI

A numeric function: π.

SYNTAX

```
PI
```

RESULT

The ratio of a circle's circumference to its diameter, as a decimal rounded to the number of significant digits that `DIGITS` has set: 3.14159265359 at the usual 12.

It takes no argument and no parentheses.

SEE ALSO

DIGITS, SIN, COS, TAN, ATN.

EXAMPLE

```
> PRINT PI : DIGITS 30 : PRINT PI : DIGITS 12
 3.14159265359
 3.14159265358979323846264338328
```

PLAY

A statement: plays music, and waits until it has finished.

SYNTAX

```
PLAY [channel,] score$
PLAY file$
```

ARGUMENTS

score$

A string expression: a line of the score language, MML (section 15.6).

channel

A numeric expression: the sound channel to play it on (section 15.1). Channel 0 if it is left out.

file$

A string expression ending in `.zsm` or `.mml`: the name of a song file, or of a file of score lines.

DESCRIPTION

HyBASIC does not make the sound itself. It runs the `play` program (section 15.7) with the string, and waits for it to end: `play -m channel score` for a line of score, and `play file` for a file. Whatever `play` can play, `PLAY` can. The program stops while the music lasts.

A string is taken as a file's name if it ends in `.zsm` or `.mml`, and as a score otherwise.

ERRORS

ERR	MESSAGE	CAUSE
511	`play`'s own message	`play` could not play it: a mistake in the score, a file that isn't there. `play` prints its message itself first, with its name before it
5	`illegal function call`	A channel number that is not one
13	`type mismatch`	The score is a number

NOTES

This is not QuickBASIC's `PLAY` language, though simple tunes look alike in both: the letters `c` to `b` are notes, `o` sets the octave and `l` the length. Section 15.6 has the whole language.

For a single tone, `SOUND` is simpler.

SEE ALSO

`SOUND`, `BEEP`, `SHELL`.

EXAMPLE

```
> PLAY "t240 l16 c e g" : PRINT "played"
played
> PLAY "zzz!"
play: zzz!: channel 0: what is z
channel 0: what is z
> PRINT ERR
 511
```

POS

A numeric function: the column the next character will be printed in.

SYNTAX

```
POS(x)
POS
```

ARGUMENTS

x

Any numeric expression. It is ignored; `POS(0)` is the usual form, from QuickBASIC, and `POS` alone means the same.

RESULT

The column, counted from 1 at the left edge. HyBASIC keeps count of what it has written since the last line end, so this is the column of its own output: after `PRINT "ab";` it is 3.

NOTES

The count is HyBASIC's, not the terminal's. It follows `LOCATE`, but it knows nothing of what other programs have written, or of escape sequences a program prints for itself. The row is `CSRLIN`, which does ask the terminal.

`TAB` and the print zones of a comma work from the same count.

SEE ALSO

`CSRLIN`, `LOCATE`, `TAB`, `SPC`, `WIDTH`.

EXAMPLE

```
> PRINT POS(0); : PRINT "ab"; : PRINT POS(0)
 1 ab 6
```

PRESERVE

Part of `REDIM`: `REDIM PRESERVE` changes an array's size and keeps what is in it. See `REDIM`.

PRINT

A statement: writes values as text, to the screen or to a file.

SYNTAX

```
PRINT [#f,] [item [{; | ,} item] ...] [; | ,]
PRINT [#f,] USING format$; item [{; | ,} item] ... [; | ,]
? ...
```

ARGUMENTS

item

An expression of any type, or `TAB(n)` or `SPC(n)`.

f

The number of a file opened `FOR OUTPUT`, `FOR APPEND` or `FOR BINARY`. The output goes to the file in place of the screen, byte for byte the same.

format$

A string expression: a picture of how the items are to be written (below).

`?` is another spelling of `PRINT`.

DESCRIPTION

Values. A string is written as its bytes. A number is written as its text in the base that `BASE` has set, with a space before it if it isn't negative (a negative number has its minus sign there instead) and a space after it. The text is the number's exact value: an integer has all its digits however many there are, a rational is written with a `/`, and a decimal with its point and every digit it holds. There is no exponent form: a very large or very small number is written out in full.

Between items. A semicolon writes nothing: the next item follows at once. A comma moves to the next print zone: the zones are 14 columns wide, starting at columns 1, 15, 29 and so on, and if there is no whole zone left on the line the item starts a new line. Two commas skip a zone.

At the end. `PRINT` ends the line, by writing a line feed, unless the statement ends with a semicolon or a comma: then the next `PRINT` carries on from there. `PRINT` with no items writes an empty line.

`TAB(n)` moves to column *n*, counting from 1, by writing spaces. If the output is already past column *n*, it starts a new line and moves to column *n* there. **`SPC(n)`** writes *n* spaces. Both are used only inside `PRINT`.

The line's width is 80 unless `WIDTH` has set another. It decides where the zones run out. Text longer than the width is not broken by HyBASIC; the terminal wraps it.

PRINT USING

The format is text with **fields** in it. Each item is written by the next field; the text between fields is written as it stands. If there are more items than fields, the format is used again from its start.

A number's field is a picture of the number:

IN THE FIELD	MEANS
`#`	A digit's place. The number is put at the right of its places, with spaces before it
`.`	The decimal point. The `#`s after it are decimal places, always filled, with zeros if need be
`,`	Before the point: put commas between each three digits
`+`	At the start: write the sign, `+` or –, always
`$$`	At the start: write a `$` just before the number
`**`	At the start: fill the spaces before the number with `*`

The number is **rounded** to the field's decimal places, a half going away from zero: 2.5 in `###` is 3, and −3.145 in `###.##` is −3.15. (This is not `CINT`'s rule, which sends a half to the even integer.) A minus sign takes a place. If the number needs more places than the field has, it is written in full with a `%` before it to show that the column is broken.

A string's field is one of three:

IN THE FIELD	MEANS
`!`	The first character of the string
`&`	The whole string
`\`, spaces, `\`	As many characters as the field is wide, counting both backslashes; a shorter string is padded with spaces

Two more things can be in a format:

IN THE FORMAT	MEANS
`{}`	The item as `PRINT` would write it, a number or a string, with no spaces added
`{`*base*`}`	A number in that base: `{x}` in hex, `{b}` in binary (a base string, section 18.4)
`_`	The next character is text, not part of a field: `_#` writes a `#`

A semicolon or a comma between the items of `PRINT USING` makes no difference: the format decides the spacing. At the end of the statement they hold the line open, as in `PRINT`.

ERRORS

ERR	MESSAGE	CAUSE
13	`type mismatch`	`USING`: a string for a number's field, or a number for a string's
5	`illegal function call`	`USING`: the format has no field in it
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR INPUT`

NOTES

`PRINT #` writes to a file exactly what `PRINT` would have put on the screen, zones and spaces included. For data that a program will read back with `INPUT #`, `WRITE #` is the better choice: it puts commas between the items and quotes round the strings.

To print a number without its spaces, use `PRINT USING "{}"`, or `LTRIM$(STR$(n))`.

SEE ALSO

`WRITE`, `WIDTH`, `LOCATE`, `POS`, `STR$`, `BASE`, `FIXED`, `OPEN`.

EXAMPLE

```
> PRINT 1; -2; 3 : PRINT "a"; "b"; 5; "c" : PRINT 1, 2, "x", -4
 1 -2  3
ab 5 c
 1             2             x             -4
> PRINT 1/3; 2.50; 2 ^ 64; 1+2i : PRINT "one"; : PRINT " line"
 1/3  2.5  18446744073709551616  1+2i
one line
> PRINT "A"; TAB(10); "B"; TAB(5); "C"; SPC(3); "D"
A         B
    C   D
> PRINT USING "###.##"; 3.14159 : PRINT USING "###.##|"; -3.145; 1234.5
  3.14
 -3.15|%1234.50|
> PRINT USING "#,###.# +##.# $$###.## **###.##"; 1234.56; 5; 12.5; 12.5
1,234.6  +5.0   $12.50 ***12.50
> PRINT USING "[!][&][\  \] {} {x} _#"; "hello"; "world"; "abcdefg"; 255; 255
[h][world][abcd] 255 FF #
```

PUT

A statement: writes a value to a `BINARY` file, at a given position.

SYNTAX

```
PUT [#]f, [position], value
```

ARGUMENTS

f

A numeric expression: the number of a file opened `FOR BINARY`.

position

A numeric expression: where to write, counted in bytes from 1 for the file's first byte. If it is left out (the two commas are still written) or is less than 1, the write starts where the last `GET` or `PUT` on the file ended.

value

An expression of either type. A variable is the usual thing, but any expression will do.

DESCRIPTION

A string is written as its bytes, as many as it has, with nothing before or after them. A number is written in the stored format, the form `MKN$` makes, which is as long as the number needs.

The bytes replace whatever was at that position. Writing past the end makes the file longer; writing at a position beyond the end leaves the gap between filled with zero bytes.

After the write, the file's position is just past what was written.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* was not opened `FOR BINARY`
61	`disk full`	No room left on the disk

NOTES

To write text to a file opened `FOR BINARY`, `PRINT #` works too, and adds its line end.

Since a number's stored form varies in length, a file of numbers written with `PUT` is read back in the same order with `GET`; a number can't be changed in place for a longer one without overwriting what follows it.

SEE ALSO

`GET`, `OPEN`, `SEEK`, `MKN$`, `CVN`, `PRINT`.

EXAMPLE

```
> OPEN "/ram/p.dat" FOR BINARY AS #1 : PUT #1, 1, "HELLO" : PUT #1, 3, "xy" : PUT #1, 9,
"!" : PRINT LOF(1)
 9
> A$ = SPACE$(9) : GET #1, 1, A$ : PRINT A$; ASC(MID$(A$, 6)) : CLOSE
HExyO! 0
```

CHAPTER 2

Commands

A command is a program in `/bin`, or one of the shell's own. Each ends with an **exit status**, which rc keeps in `$status`: empty for success, and a short text for failure. Most commands fail with the status `1`, and with `usage` when their arguments make no sense. A command's messages go to standard error, as `name: what: why`, so they aren't mixed into a pipe's data.

A path in an argument can be at most 63 characters long once it has been made absolute.

cp

A program (`/bin/cp`): copies files, and with `-r` directories.

SYNTAX

```
cp [-r] from to
cp [-r] from ... dir
```

ARGUMENTS

-r

Copy directories too: each one's whole tree.

from

A file to copy, or with `-r` a directory. There can be several if the last name is a directory.

to

The copy's name. If it is an existing directory, the copy goes into it under *from*'s own name.

dir

An existing directory. Each *from* is copied into it under its own name.

DESCRIPTION

Each *from* is handled in turn, and one that fails doesn't stop the rest.

A file is copied by opening it for reading, creating the destination, and reading and writing 512 bytes at a time until a read returns nothing. Creating the destination empties a file that is already there. The copy is a new file: it has its own date, the time of copying, and is not read-only even if the original is.

Before copying, `cp` compares the two files' device, disk and file identifier. If they are the same file, by whatever two names, it refuses to copy it onto itself, which would otherwise empty it.

Anything that can be opened and read can be a *from*, a device included: `cp /dev/time now` copies what one read of the clock gives.

Without `-r`, a *from* that is a directory is reported and skipped. With `-r`, the directory is made at the destination (one already there is used as it is), and then everything in it is copied in the same way, down to 16 directories deep.

EXIT STATUS

Empty if every file was copied. `1` if any was not. `usage`, after a line saying how `cp` is used, if there are fewer than two names or an option other than `-r`.

ERRORS

Each is printed as `cp: name: reason`, where *name* is the file that could not be read or written.

REASON	CAUSE
`not found`	*from* isn't there, or a directory in *to*'s path isn't
`is a directory`	*from* is a directory, and there's no `-r`
`not a directory`	Several *from*s, and the last name isn't a directory
`the same file`	*from* and *to* are one file
`read-only`	The destination's disk can't be written (`/rom`)
`name too long`	The destination's path would pass 63 characters

Any other reason is the system's own error message for what failed (Appendix G): a full disk, for one.

NOTES

If a read or a write fails part-way, the destination is left holding what was written up to then.

`cp` doesn't ask before it overwrites.

SEE ALSO

`mv`, `rm`, `mkdir`, `cat`, `cmp`.

EXAMPLE

```
% echo hello >a.txt
% cp a.txt b.txt; cat b.txt
hello
% mkdir d; cp a.txt b.txt d; ls d
a.txt
b.txt
% cp d e; echo $status
cp: d: is a directory
1
% cp -r d e; ls e
a.txt
b.txt
% cp a.txt a.txt
cp: a.txt: the same file
% cp a.txt /rom/x
cp: /rom/x: read-only
% cp a.txt; echo $status
usage: cp [-r] from to, or cp [-r] from ... dir
usage
```

CHAPTER 3

System Calls

A system call is a `jsr` to a fixed address in the jump table at `$F800`. Its arguments go in A, X, Y and the call registers r0 to r15 (two bytes each, in the zero page from `$02`); its results come back in the same places, a 16-bit result in A (low byte) and X (high byte). On return, C = 0 means success. C = 1 means failure, and A holds an error code (Appendix G). A call may change A, X, Y, the flags and r0 to r15, including registers it doesn't use for a result; it never changes the program's own zero page, `$22` to `$7F`.

A call that *can wait* may suspend the task until something happens elsewhere: a key, a disk, another task. A note delivered to the task ends such a wait, and the call returns `E_INTR`.

Each call's name is defined in `hydra.inc` (`/lib/as`), with the constants and error codes used here.

OPEN

System call `$F8C0`: opens a file that exists, and returns a file descriptor for it. It can wait.

SYNTAX

```
            LDR r0, path        ; r0 = the address of the path
            lda #mode
            jsr OPEN
            bcs failed          ; A = the error code
            sta fd              ; A = the file descriptor
```

ARGUMENTS

r0

The address of the path: its bytes, then a zero byte.

A

The mode: one of `O_READ` (0), `O_WRITE` (1) or `O_RDWR` (2), to which can be added `O_TRUNC` (`$10`), to empty the file as it is opened, and `O_NONBLOCK` (`$40`), to make reads and writes of this descriptor return `E_AGAIN` where they would otherwise wait.

RESULT

C = 0, and A = the file descriptor: a number from 0 to 15, the lowest one the task has free.

DESCRIPTION

1. The path is made absolute and clean. One that doesn't start with `/` (or with `#`, a device's own name) has the task's current directory put before it; then `.` elements and doubled slashes are removed, and each `..` removes the element before it. The result may be at most 63 bytes (`PATH_MAX`).
2. The path is looked up in the task's namespace (section 9.5). Where a directory is a union of several (section 9.7), each member is tried in turn, and the first that has the name is the one opened. A member whose device has gone is passed over as if it didn't have the name.
3. The device's driver is asked to open the file in the mode given. If it refuses, its error is the call's.
4. The descriptor is set to the file, at offset 0.

A descriptor has an offset of its own, which `READ`, `WRITE` and `SEEK` move. Descriptors made from it by `DUP`, and those a child task is given by `SPAWN`, share that offset; a second `OPEN` of the same file does not.

Opening a directory with `O_READ` succeeds: it reads as a sequence of 64-byte stat records, one for each file in it. When the directory is a union, the records of all its members follow one another.

`OPEN` never makes a file. `CREATE` does, and opens it.

ERRORS

CODE	NAME	CAUSE
`$20`	`E_NOENT`	No such file, in any member of a union
`$2B`	`E_NODEV`	The path names a device that isn't there
`$0A`	`E_NAMETOOLONG`	The path, made absolute, is longer than 63 bytes
`$26`	`E_MFILE`	The task has all 16 of its descriptors in use
`$27`	`E_NFILE`	All 48 of the system's channels are in use (every open file in every task takes one; `DUP`s share)
`$05`	`E_INTR`	A note arrived while the call was waiting for the driver

A driver adds its own: `E_ROFS` (`$29`) for a write mode on a read-only disk, and `E_ISDIR` (`$23`) for a write mode on a directory.

NOTES

Descriptors 0, 1 and 2 are standard input, output and error, already open when a program starts. A task that closes one of them will get that number from its next `OPEN`.

A descriptor that isn't closed is closed when the task ends.

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-open` (*z-addr mode -- fd ior*), the path zero-terminated (`>z` makes one)
hylang	`(sys-open path [mode])`: the descriptor. A failure is an error, its text the path and the system's message
HyBASIC	`SYS "OPEN"`; or, for most purposes, the `OPEN` statement (Chapter 1)

SEE ALSO

`CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `DUP`, `FSTAT`.

EXAMPLE

From hylang. Descriptors 0 to 2 are in use, so the first file opened is 3:

```
hylang> (sys-open "/dev/time" 0)
=> 3
hylang> (sys-open "/dev/nothing" 0)
=> Error: /dev/nothing: not found
hylang> (sys-open "/rom/bin/ls" 1)
=> Error: /rom/bin/ls: read-only
hylang> (sys-open "/ram" 1)
=> Error: /ram: is a directory
hylang> (sys-open "#q/x" 0)
=> Error: #q/x: no such device
```

READ

System call `$F8C9`: reads bytes from an open file into memory. It can wait.

SYNTAX

```
            LDR r0, buf         ; r0 = where the bytes go
            LDR r1, count       ; r1 = how many, at most
            lda fd
            jsr READ
            bcs failed          ; A = the error code
                                ; A/X = how many were read; 0 is the end of the file
```

ARGUMENTS

A

A file descriptor, open with `O_READ` or `O_RDWR`.

r0

The address of a buffer in the task's memory.

r1

The most bytes to read: 0 to 65,535. The buffer must have room for them.

RESULT

C = 0, and A/X = the number of bytes read, which were put in the buffer from its start. The descriptor's offset has moved on by that many.

DESCRIPTION

The kernel asks the file's driver for the bytes at the descriptor's offset, at most 512 at a time (`IO_UNIT`), and repeats until one of three things happens:

- **The count is reached.** The result equals r1.
- **The file ends.** The result is what there was before the end; a read that starts at the end returns 0. A result of 0 is how the end of a file is recognised, and it is not an error.
- **The driver returns fewer bytes than it was asked for.** The kernel stops there and returns them. This is how a device says "that is all for now": the console returns one line, however large the buffer.

A result smaller than r1 therefore does not by itself mean the end of the file. A program that needs exactly *n* bytes must call `READ` again for the remainder until it has them or gets 0.

When the driver has nothing yet (no key typed, an empty pipe), the task waits. With `O_NONBLOCK` set on the descriptor, the call returns `E_AGAIN` instead.

A directory reads as 64-byte stat records (`SR_SIZE`), one for each file in it: ask for a multiple of 64.

ERRORS

CODE	NAME	CAUSE
`$25`	`E_BADF`	A isn't an open descriptor, or it was opened with `O_WRITE`
`$04`	`E_AGAIN`	Nothing to read yet, and the descriptor is non-blocking
`$05`	`E_INTR`	A note arrived while the call was waiting

A driver adds its own: a disk that fails answers `E_IO` (`$2A`).

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-read` (*fd c-addr u -- u2 ior*): *u2* bytes were read
hylang	`(sys-read fd count)` : the bytes read, as a string
HyBASIC	`SYS "READ"`; or `INPUT #`, `LINE INPUT #`, `INPUT$` and `GET`

SEE ALSO

`OPEN`, `WRITE`, `SEEK`, `CLOSE`, `PIPE`.

EXAMPLE

The offset moving on, a count cut short by the end of the file, and 0 at the end:

```
hylang> (set {fd} (sys-open "/ram/t" 0))
=> NIL
hylang> (sys-read fd 5)
=> "hello"
hylang> (sys-read fd 100)
=> " world\n"
hylang> (sys-read fd 100)
=> ""
hylang> (sys-close fd)
=> NIL
```

CHAPTER 4

HyForth Words

A cell is 16 bits. A double is two cells, 32 bits, its high cell on top of the stack. A flag is 0 for false and –1 (all bits set) for true. *ior* is 0 for success and a non-zero error code otherwise. Stack pictures follow the Forth standard's notation: *n* signed, *u* unsigned, *d* and *ud* doubles, *c-addr u* the address and length of a string.

Each entry says which word set a word belongs to, and whether it is in the ROM core or needs a library loaded with `lib` first. Where a word is the standard's, the entry describes what HyForth does in the cases the standard leaves open.

>number

Core word set, in the ROM core. Converts the digits at the start of a string, adding them to a double.

SYNTAX

```
>number  ( ud1 c-addr1 u1 -- ud2 c-addr2 u2 )
```

ARGUMENTS

ud1

The number so far: 0 to start a new one.

c-addr1 u1

The text to convert.

RESULT

ud2

ud1 with each converted digit added: for every digit, *ud* × `base` + the digit's value.

c-addr2 u2

What was not converted: the address of the first character that isn't a digit and the number of characters from there to the end. *u2* is 0 if the whole string was digits, and *c-addr2* is then the address just past it.

DESCRIPTION

Characters are taken from the left for as long as each is a digit in the current `base`. The digits are `0` to `9`, then the letters `A` to `Z` for 10 to 35; small letters count as capitals. A character whose value is not less than `base` is not a digit, and conversion stops at it: in decimal `a` stops it, in hex `g` does.

Nothing else is understood. A sign, a prefix (`$`, `#` or `%`) and a decimal point all stop the conversion like any other non-digit; an empty string converts nothing and returns *ud1* unchanged. Signs and prefixes are the text interpreter's business: it deals with them itself, and calls this word's code for the digits.

ERRORS

None. A string with no digits is not an error: *ud2* = *ud1* and *u2* = *u1*.

NOTES

A value that passes 32 bits wraps: only its low 32 bits are kept, and no error is raised. (The standard calls this an ambiguous condition.) To read a number of any length, use the numbers library (`lib numbers`).

`base` can be 2 to 36.

SEE ALSO

`base`, `decimal`, `hex`, `s>d`, `evaluate`.

EXAMPLE

`nip` drops the address, leaving the double (its low cell first, as `.s` shows it) and the count of characters not converted:

```
/> 0 0 s" 123abc" >number nip .s
<3> 123 0 3
/> drop 2drop
/> hex 0 0 s" ff zz" >number nip .s decimal
<3> FF 0 3
/> drop 2drop
/> 0 0 s" -5" >number nip .s
<3> 0 0 2
/> drop 2drop
/> 0 0 s" 99999999999" >number nip .s
<3> 2839 -20518 0
```

CHAPTER 5

hylang Forms

An expression in parentheses is evaluated: its first element names a function or a special form, and the rest are its arguments. A function's arguments are evaluated before it is called, left to right. A **special form** receives its arguments as they were written, and decides for itself which to evaluate and when; each entry for one says exactly that. An expression in braces, a Q-expression, is data: it is not evaluated until something runs it.

A name is looked up in the current **scope**, then in the scope that encloses it, and so on out to the global scope. Each call of a function has a scope of its own, enclosed by the scope the function was made in.

An error is a value: `Error:` and a message. An expression that is given an error as an argument returns it at once, so an error passes outward through whatever contains it until something catches it or it reaches the prompt.

`let`

A special form: evaluates expressions in a new scope, with names bound in it first.

SYNTAX

```
(let {{name [value]} ...} body ...)
(let body)
```

ARGUMENTS

name

A symbol.

value

An expression: what the name is bound to. If it is left out, the name is bound to `NIL`.

body

An expression. It can also be written as a Q-expression, `{print x}`, which is then run as code.

RESULT

The value of the last *body* expression.

DESCRIPTION

1. A new scope is made, enclosed by the scope the `let` is in.

2. The bindings are taken in order. For each, *value* is evaluated **in the new scope**, and *name* is bound to the result there. A value can therefore use the names bound before it in the same `let`; a name bound twice keeps its last value.
3. The *body* expressions are evaluated in order, in the new scope. The values of all but the last are discarded.
4. The new scope is dropped, unless something made in it still refers to it: a function made by `fn` in the body keeps the scope, and can go on using its names after the `let` has returned (a closure).

A name bound by `let` hides one of the same name outside it for as long as the body runs, and the outer one is unchanged afterwards.

Inside the body, `set` binds or changes a name in the `let`'s own scope; `set!` changes a name in whichever scope it was made, which may be outside; and `def` always binds globally.

The last body expression is evaluated in tail position: a function whose body ends in a `let` that ends in a call to itself runs in constant stack, however many times it repeats.

With one argument only, that argument is the body: `(let body)` evaluates it in a new, empty scope. This means `(let {{a 1}})` does not bind anything; its one argument is run as code, and the result is the list `{a 1}`.

ERRORS

MESSAGE	CAUSE
`A 'let' binding is {name value}`	A binding that isn't a list, whose first element isn't a symbol, or that has more than two elements
`'let' supplied too few parameters`	No arguments at all

An error from evaluating a *value* or a *body* expression ends the `let` at once, and is its result.

SEE ALSO

`def`, `set`, `set!`, `fn`, `do`.

EXAMPLE

```
hylang> (let {{x 2} {y (+ x 1)}} (list x y))
=> {2 3}
hylang> (def {x} 10)
=> NIL
hylang> (let {{x 1}} (set {x} 5) x)
=> 5
hylang> x
=> 10
hylang> (def {add5} (let {{n 5}} (fn {a} {+ a n})))
=> NIL
hylang> (add5 1)
=> 6
hylang> n
=> Error: Unbound Symbol 'n'
hylang> (let {a 1} a)
=> Error: A 'let' binding is {name value}
```